



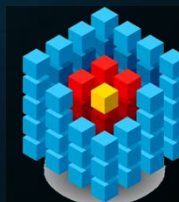
Inteligencia Artificial y Aprendizaje Automático

Sesión 3: Aprendizaje profundo

Presentan:

Dr. Ulises Olivares Pinto

Fernando Ramírez González



HPC
DATA LAB

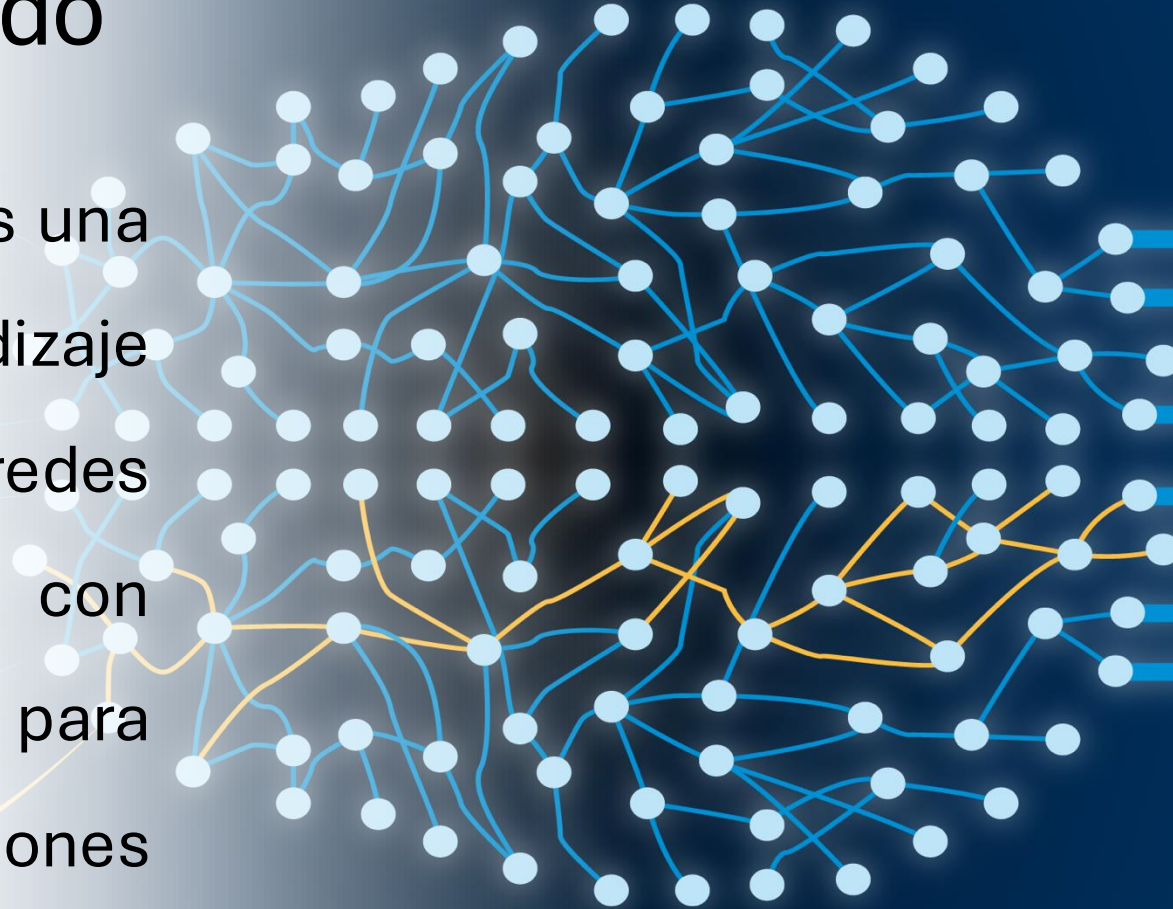
Contenido

Aprendizaje profundo

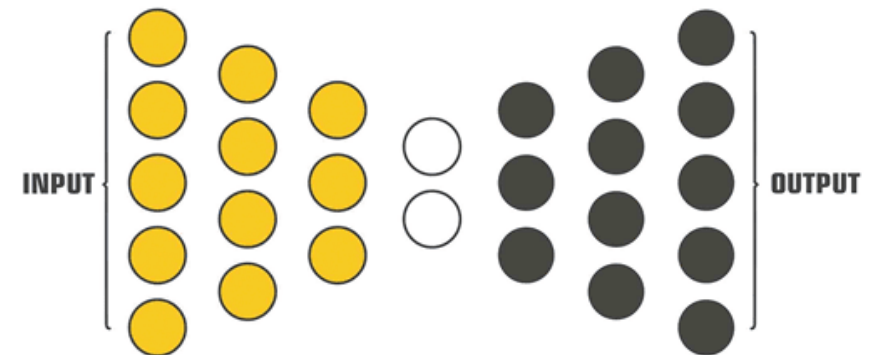
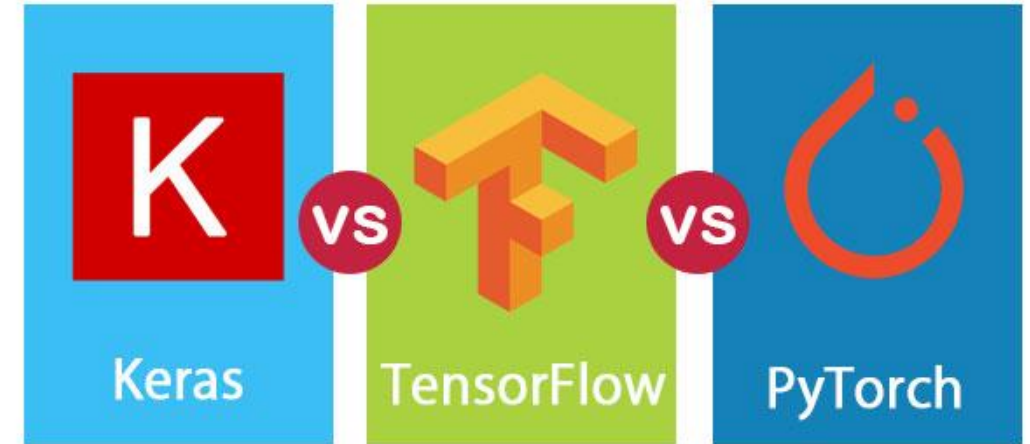
- Definiciones
- Perceptrón
- Funciones de activación
- Perceptrón multicapa
- Propagación
- Entrenamiento

Aprendizaje Profundo

El **aprendizaje profundo** es una rama del aprendizaje automático que utiliza redes neuronales artificiales con muchas capas (profundas) para aprender representaciones complejas de datos.



Librerías Para Aprendizaje profundo

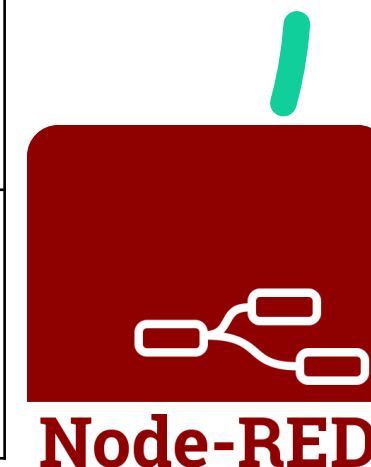




Herramientas No-Code

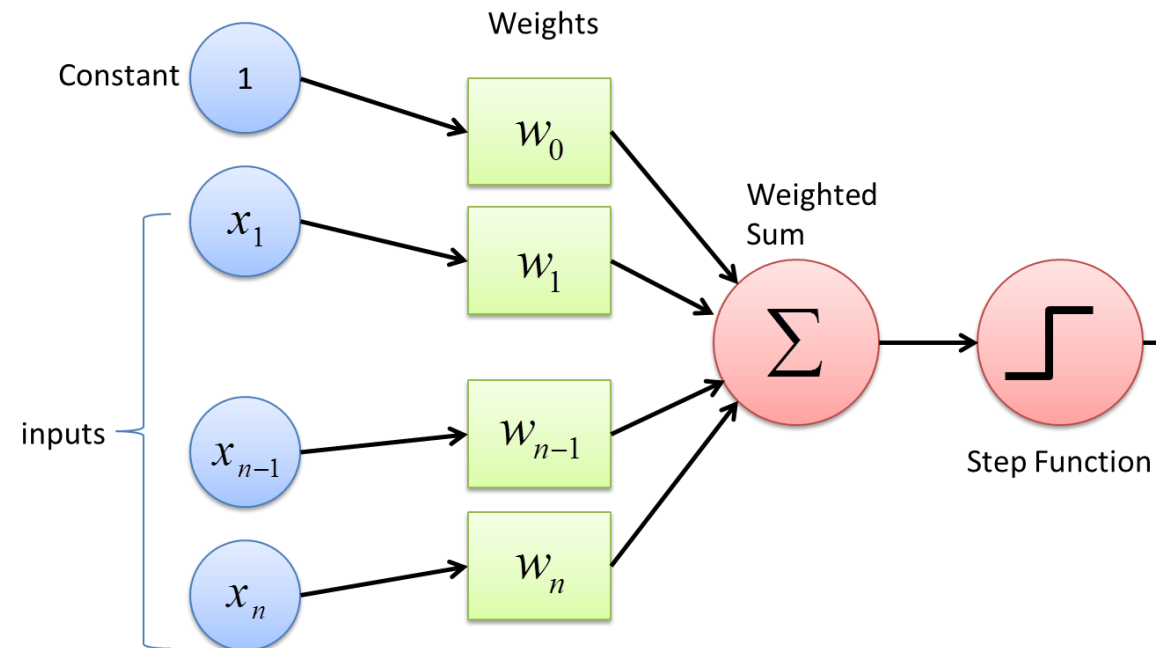
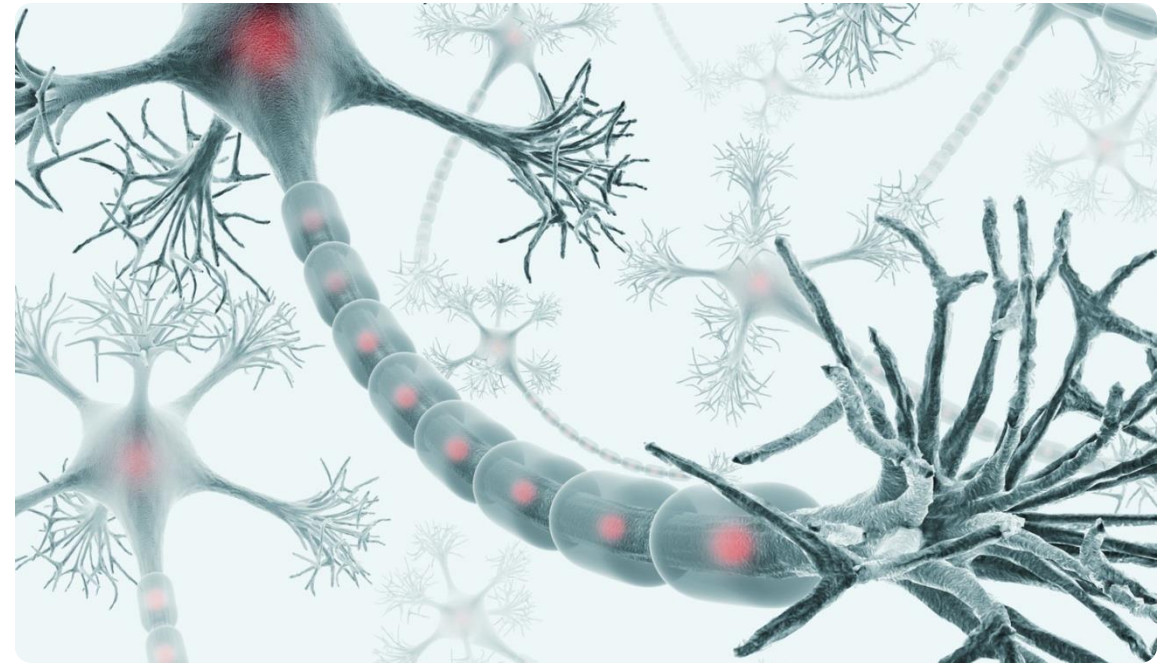


Herramienta	Descripción breve	Sitio oficial
KNIME	Plataforma de flujos de trabajo para análisis de datos, minería de datos y ML.	knime.com
Orange	Entorno visual para minería de datos, machine learning y visualización interactiva.	orangedatamining.com
Apache Nifi	Automatización de flujos de datos (Data Flow) muy flexible. Ideal para ETL visual.	nifi.apache.org
Node-RED	Plataforma de flujos de trabajo visual para integrar APIs, dispositivos y servicios web.	nodered.org
Apache Airflow	Orquestación de flujos de trabajo (no-code/low-code para pipelines de datos complejos).	airflow.apache.org



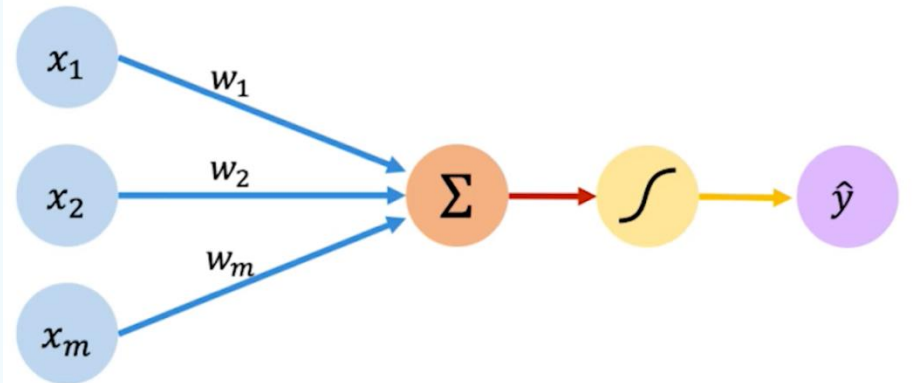
Perceptrón

El modelo fundamental (básico)
para Deep Learning



Perceptrón: Propagación hacia delante (forward propagation)

Una neurona simple



Inputs Weights Sum Non-Linearity Output

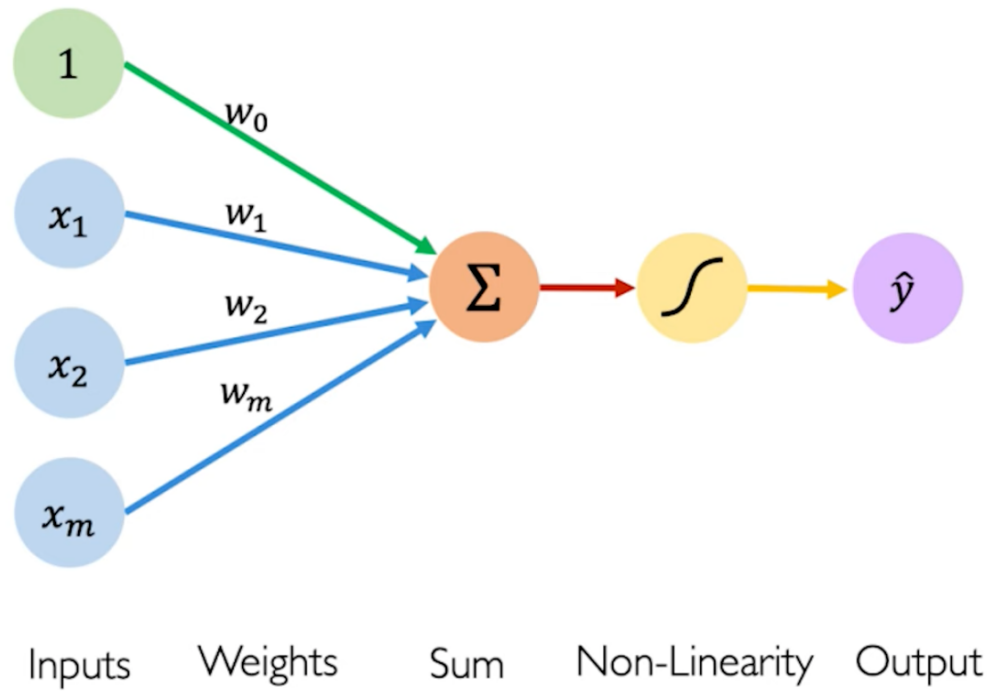
Output

Linear combination of inputs

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Perceptrón: Propagación hacia delante (forward propagation)



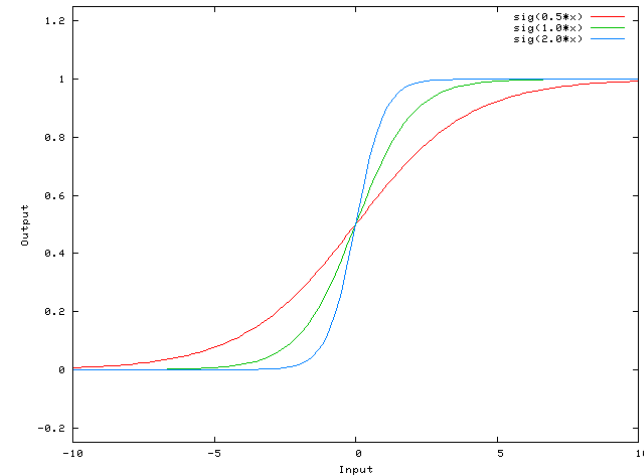
Output

Linear combination of inputs

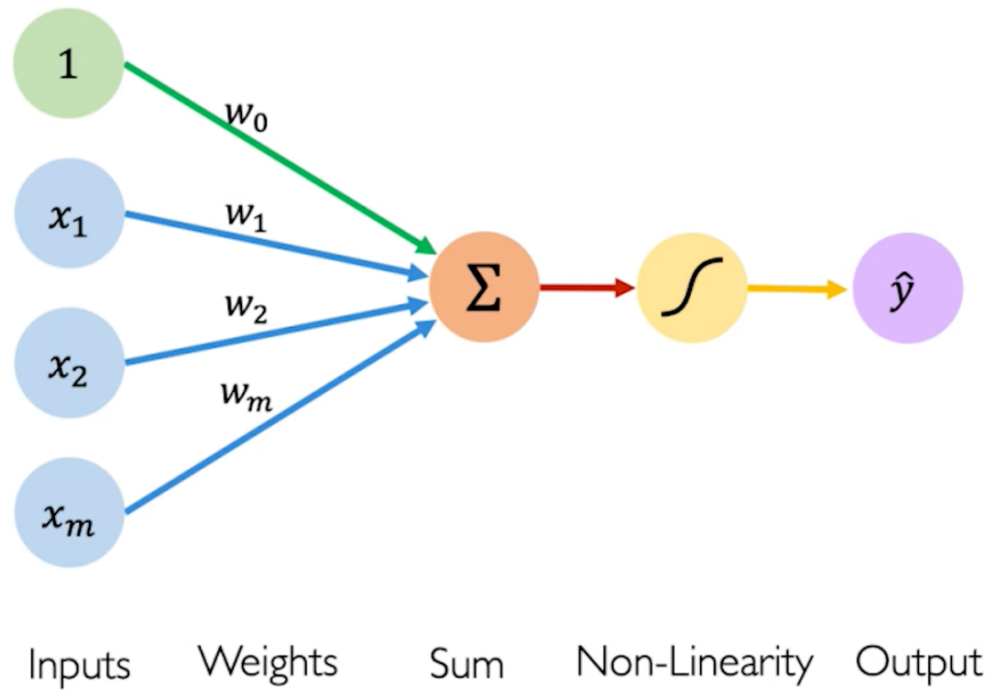
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Bias



Perceptrón: Propagación hacia delante (forward propagation)



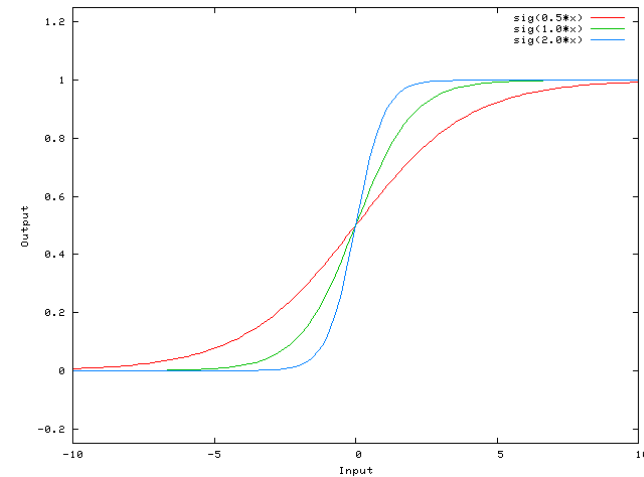
Output

Linear combination of inputs

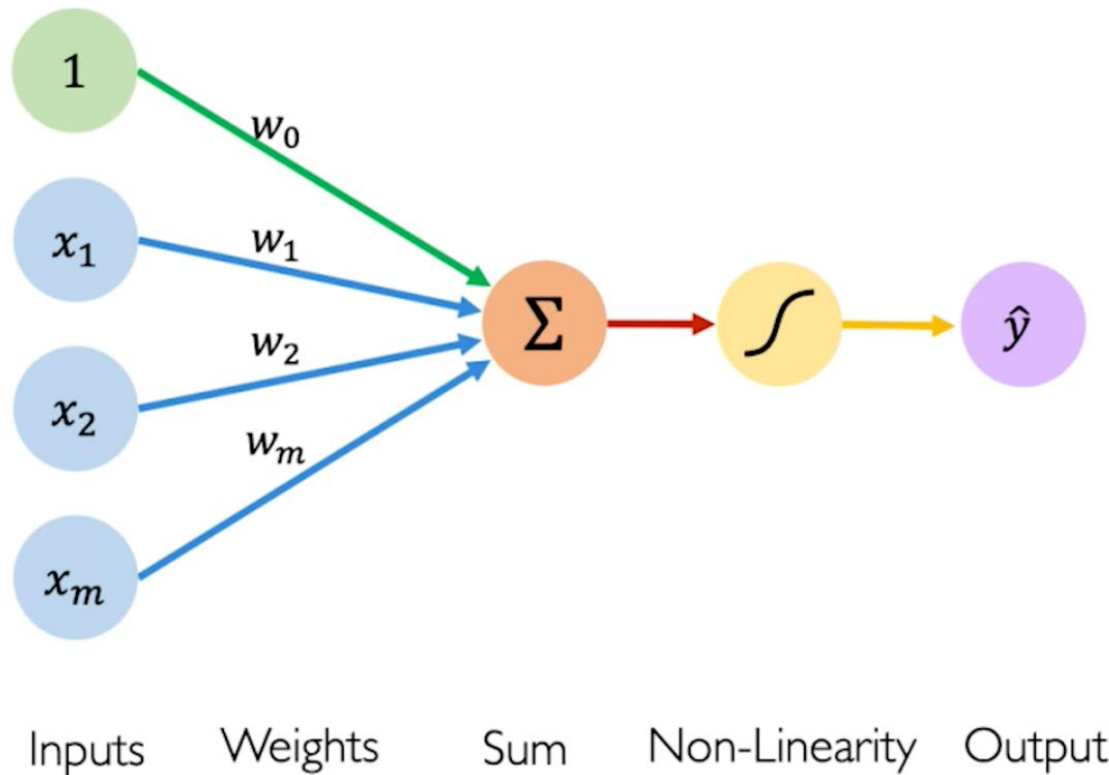
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Bias



Perceptrón: Propagación hacia delante - bias (forward propagation)

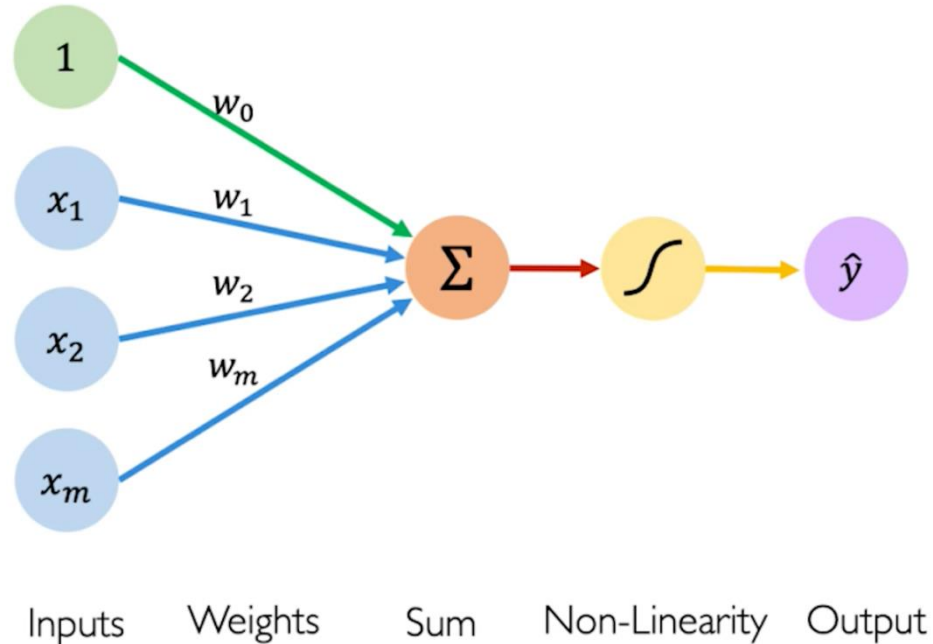


$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

$$\hat{y} = g (w_0 + \mathbf{X}^T \mathbf{W})$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Perceptrón: Propagación hacia delante - bias (forward propagation)

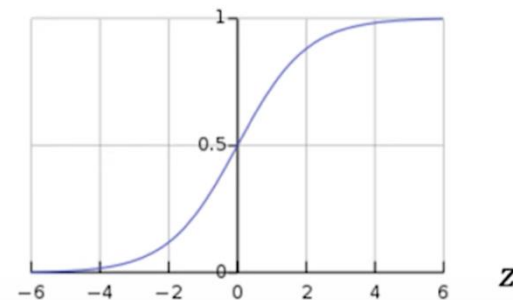


Activation Functions

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

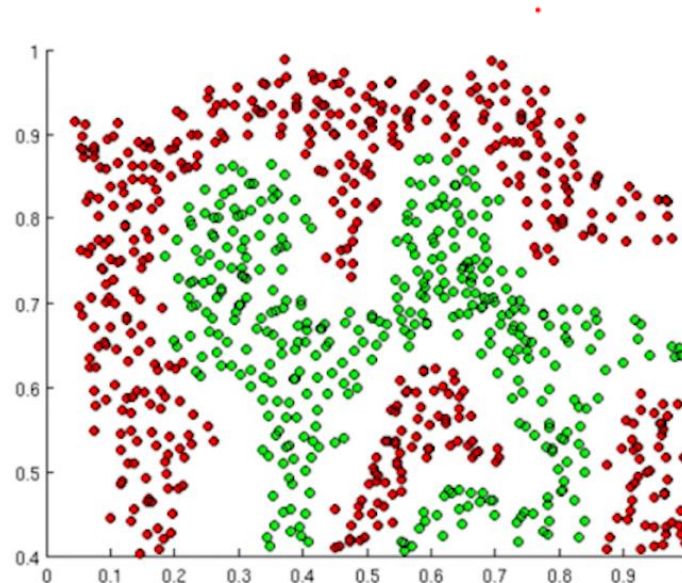
- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



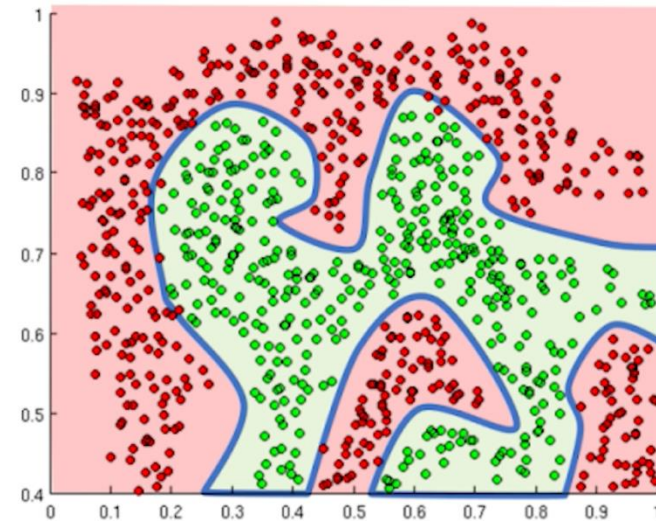
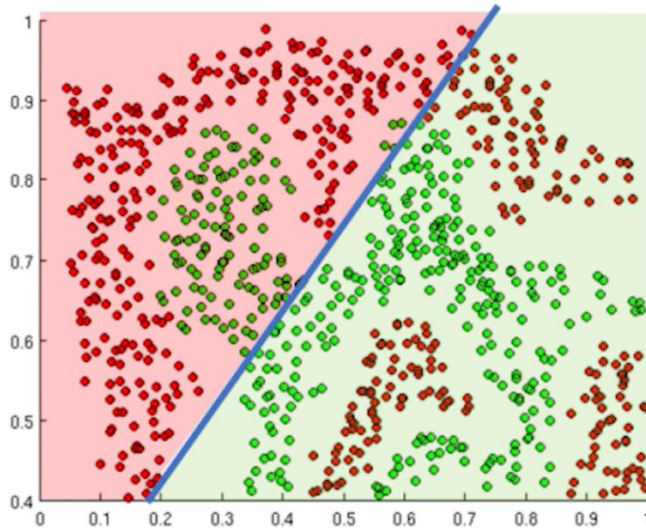
¿Por qué Funciones de activación?

- Introducen no linealidad en la Red Neuronal (RN).
 - **Problema:** Es necesario diferenciar los puntos **rojos** de los puntos **verdes**. Se deberá usar una función lineal.



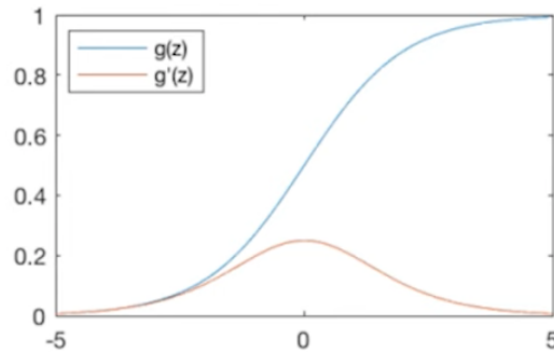
¿Por qué Funciones de activación?

- Introducen no linealidad en la Red Neuronal (RN).
 - **Problema:** Es necesario diferenciar los puntos **rojos** de los puntos **verdes**. Se deberá usar una función lineal.



Funciones más comunes de activación (no lineales)

Sigmoid Function

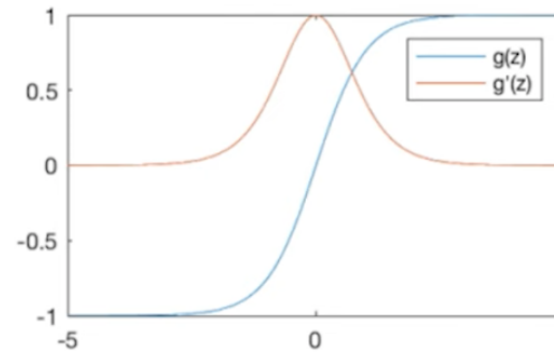


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

 `tf.math.sigmoid(z)`

Hyperbolic Tangent

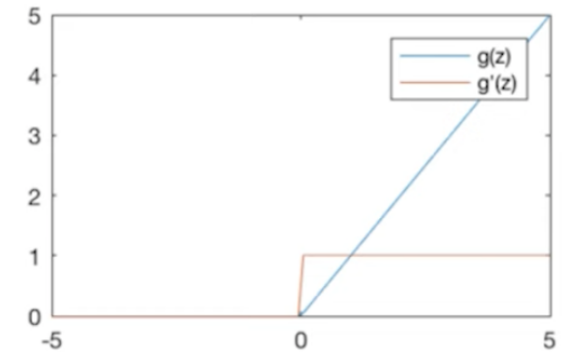


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

 `tf.math.tanh(z)`

Rectified Linear Unit (ReLU)



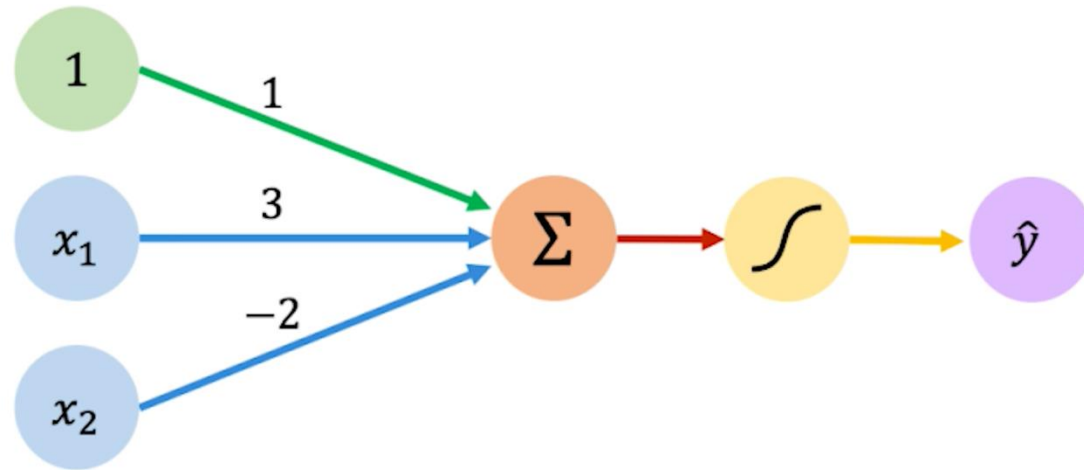
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

 `tf.nn.relu(z)`

Perceptrón: Ejemplo

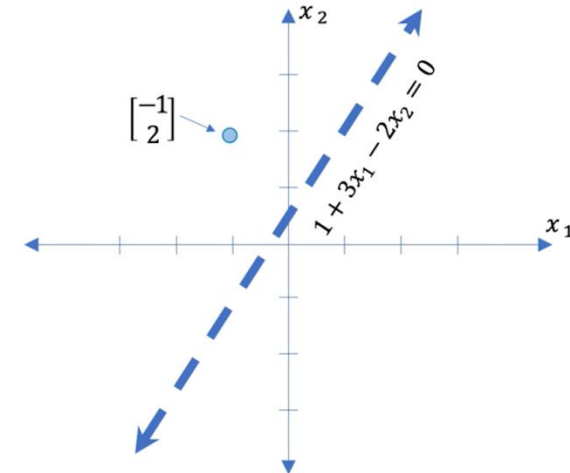
$$w_0 = 1 \text{ and } \mathbf{W} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$



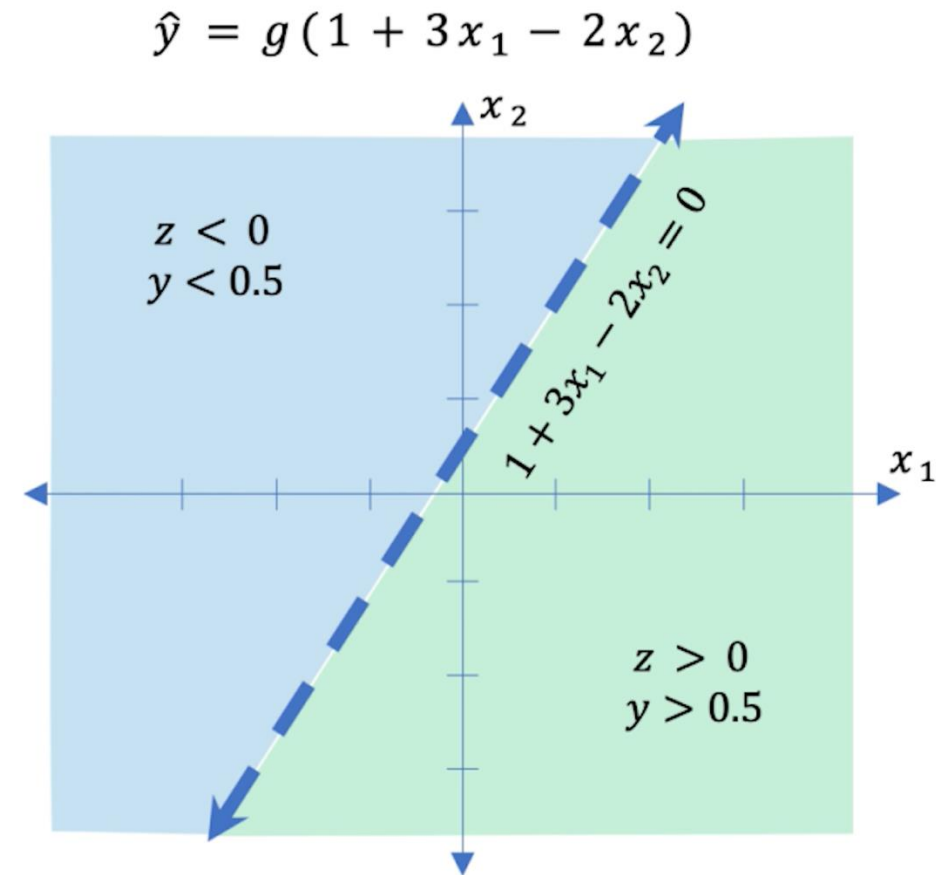
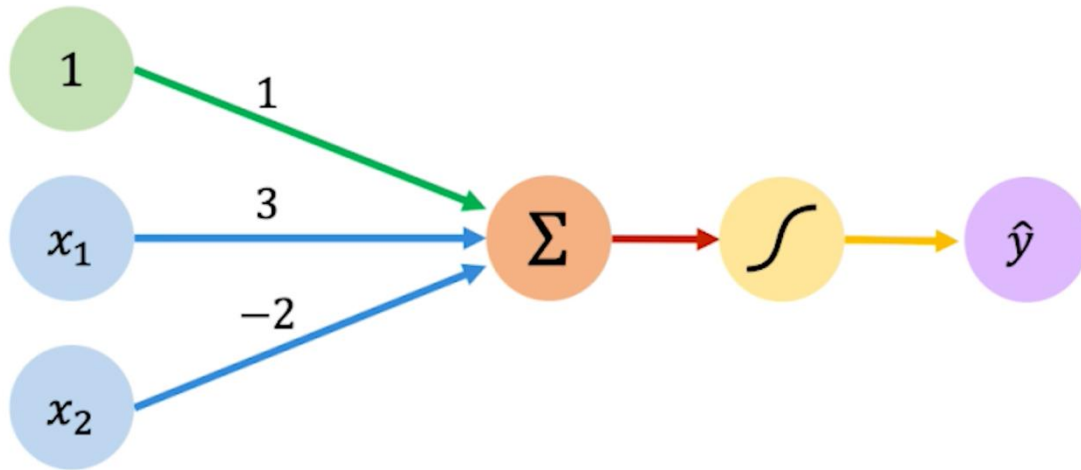
$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

$$\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$\begin{aligned}\hat{y} &= g(1 + (3 * -1) - (2 * 2)) \\ &= g(-6) \approx 0.002\end{aligned}$$



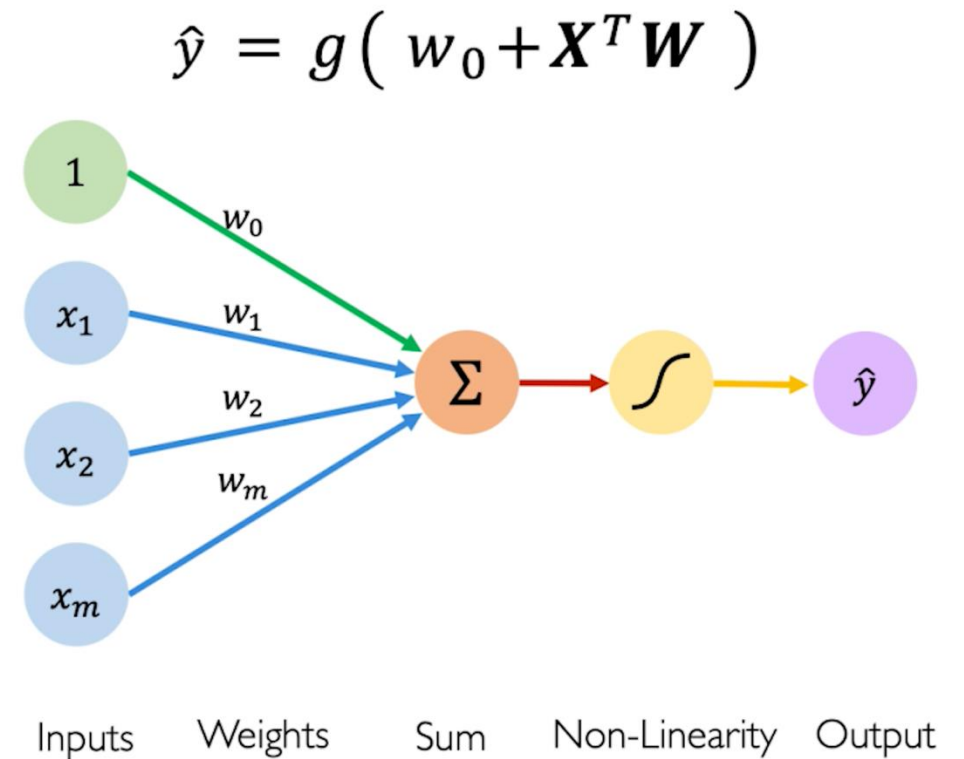
Perceptrón: Ejemplo



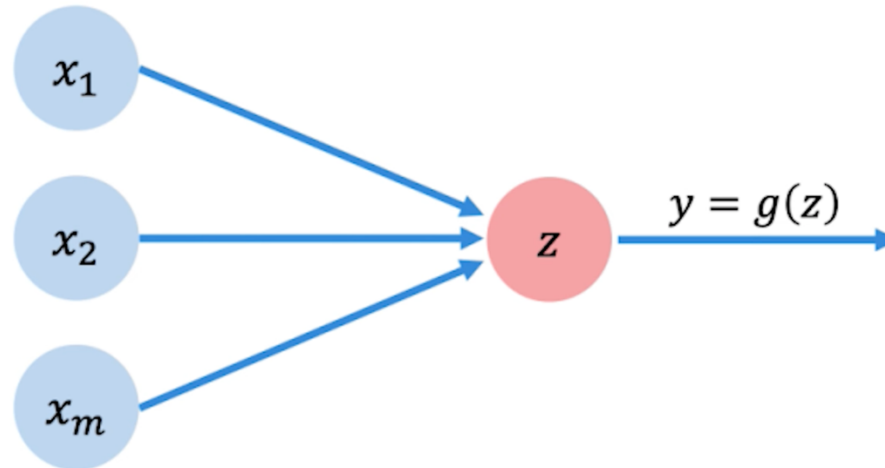
Perceptrón Simplificado

Tres pasos importantes:

1. Producto punto
2. Bias
3. Función no lineal



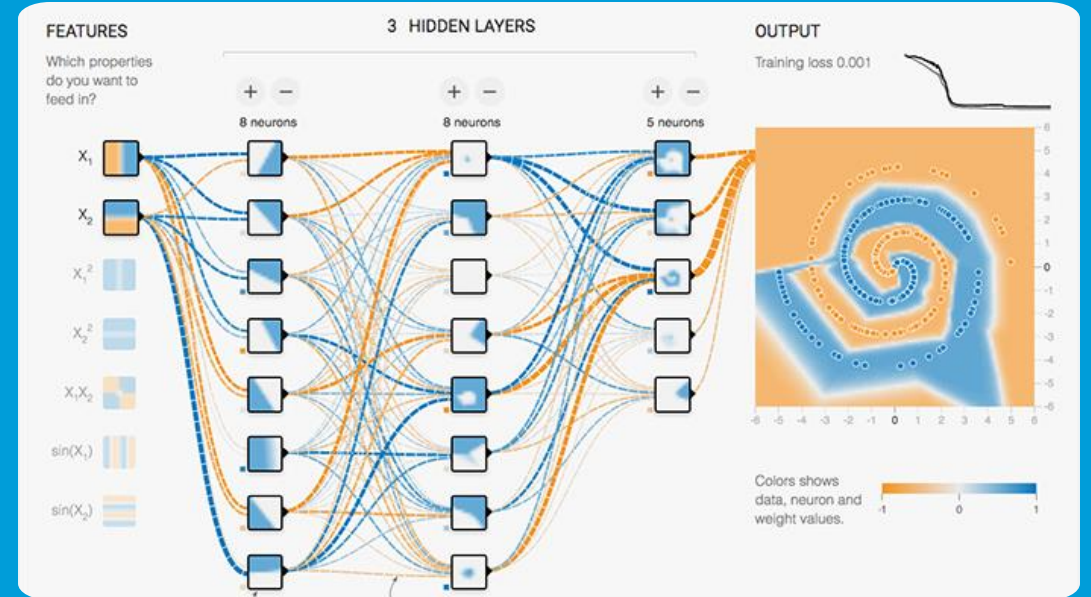
Perceptrón Simplificado



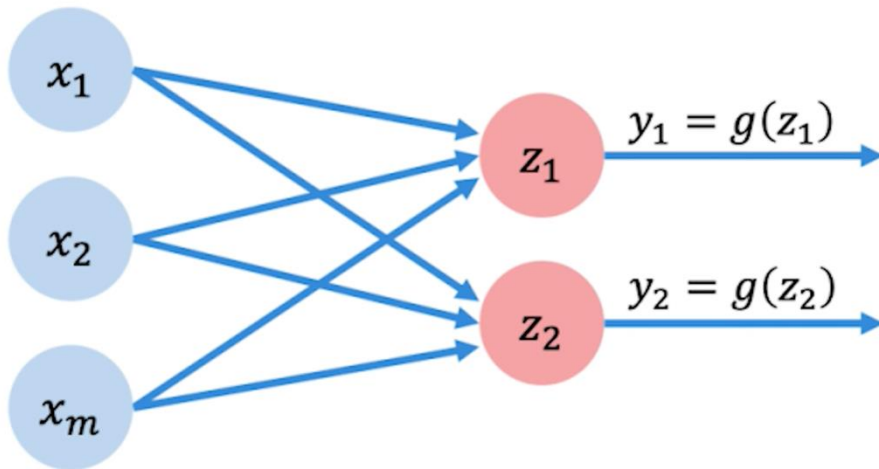
$$z = w_0 + \sum_{j=1}^m x_j w_j$$

Demos

<https://playground.tensorflow.org/>



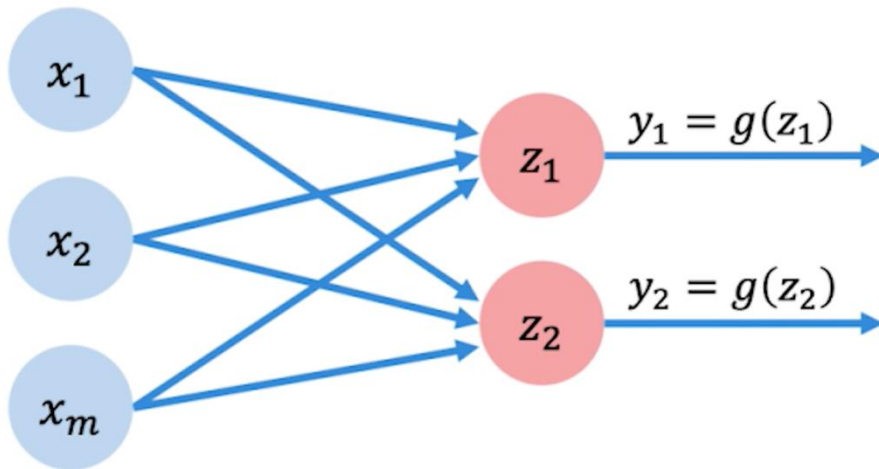
Perceptrón con múltiples salidas



Debido a que todas las **entradas** están **densamente conectadas** con todas las **salidas**. A estas capas se les denomina **capas densas**.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

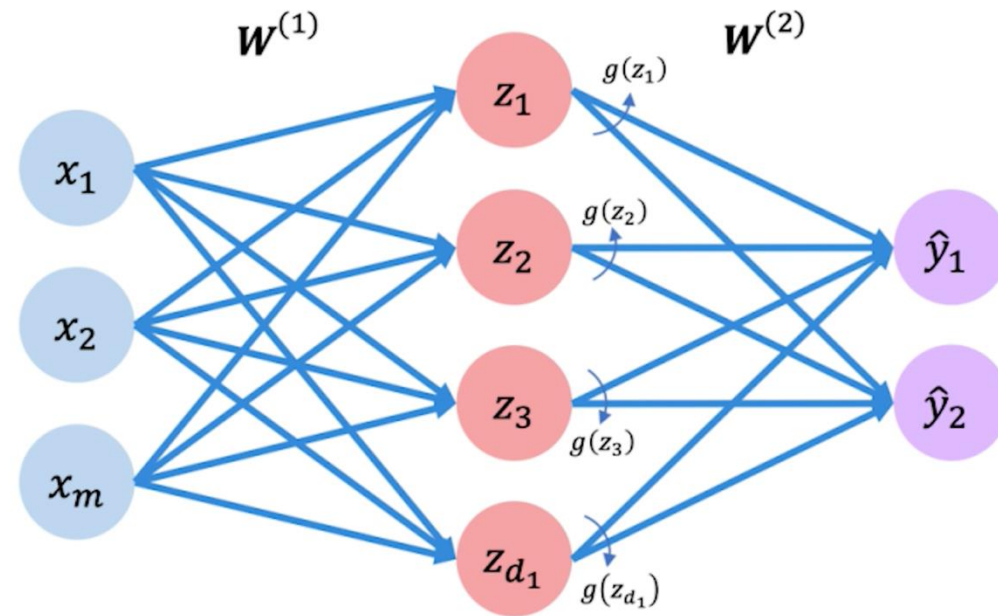
Perceptrón con múltiples salidas



```
import tensorflow as tf  
  
layer = tf.keras.layers.Dense(  
    units=2)
```

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Red Neuronal con una sola Capa (oculta)



Inputs

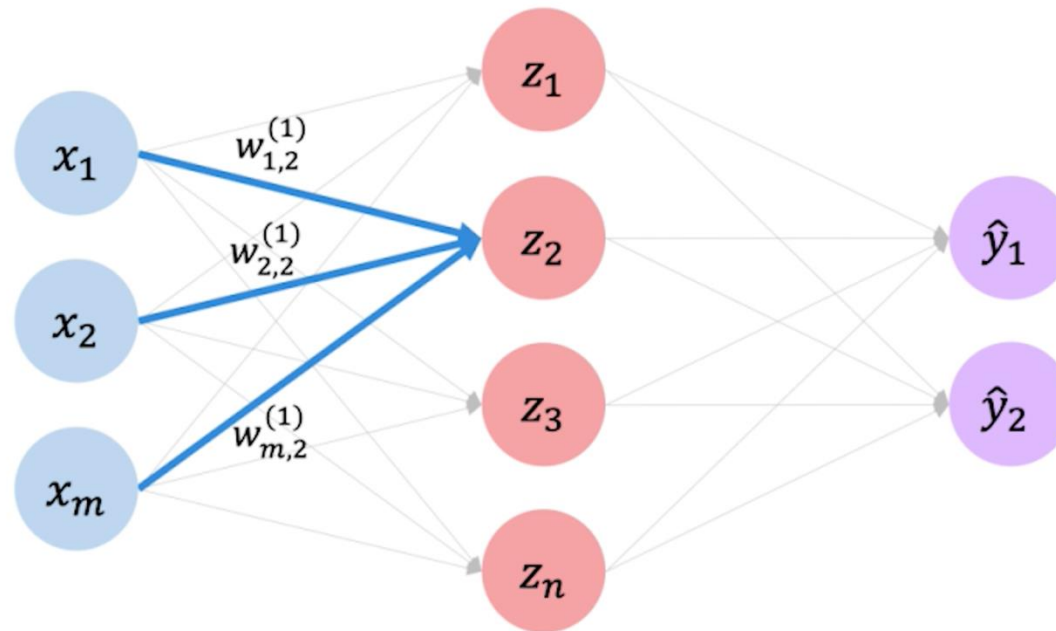
Hidden

Final Output

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

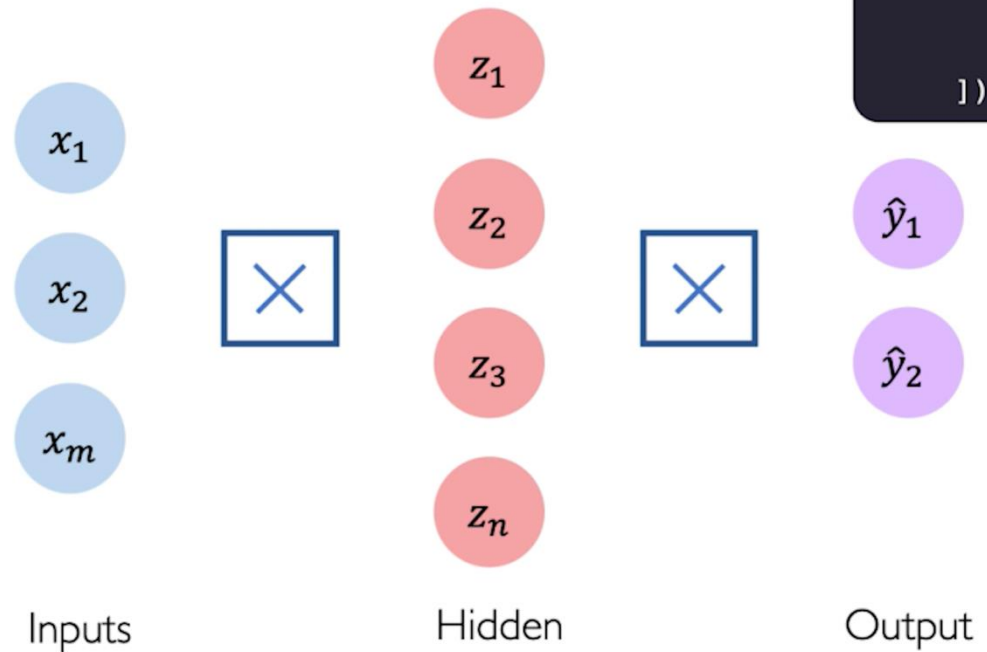
$$\hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

Red Neuronal con una solo capa oculta



$$\begin{aligned} z_2 &= w_{0,2}^{(1)} + \sum_{j=1}^m x_j w_{j,2}^{(1)} \\ &= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)} \end{aligned}$$

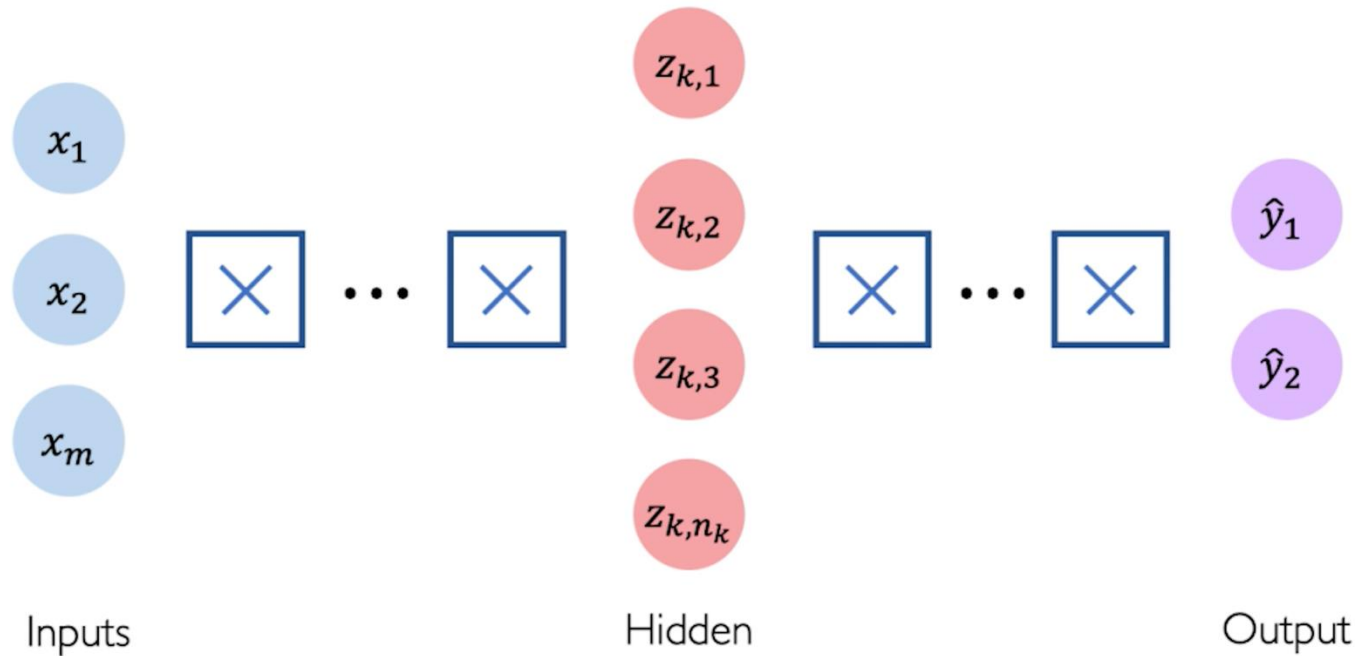
Red Neuronal con una solo capa oculta



```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n),
    tf.keras.layers.Dense(2)
])
```

Red Neuronal Profunda



$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$



```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n_1),
    tf.keras.layers.Dense(n_2),
    :
    tf.keras.layers.Dense(2)
])
```

Problema de Aplicación

Redes Neuronales

Problema de aplicación

Es posible predecir si un estudiante pasará o no un curso:

Se asumirá un modelo muy simple con dos características

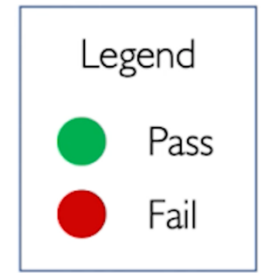
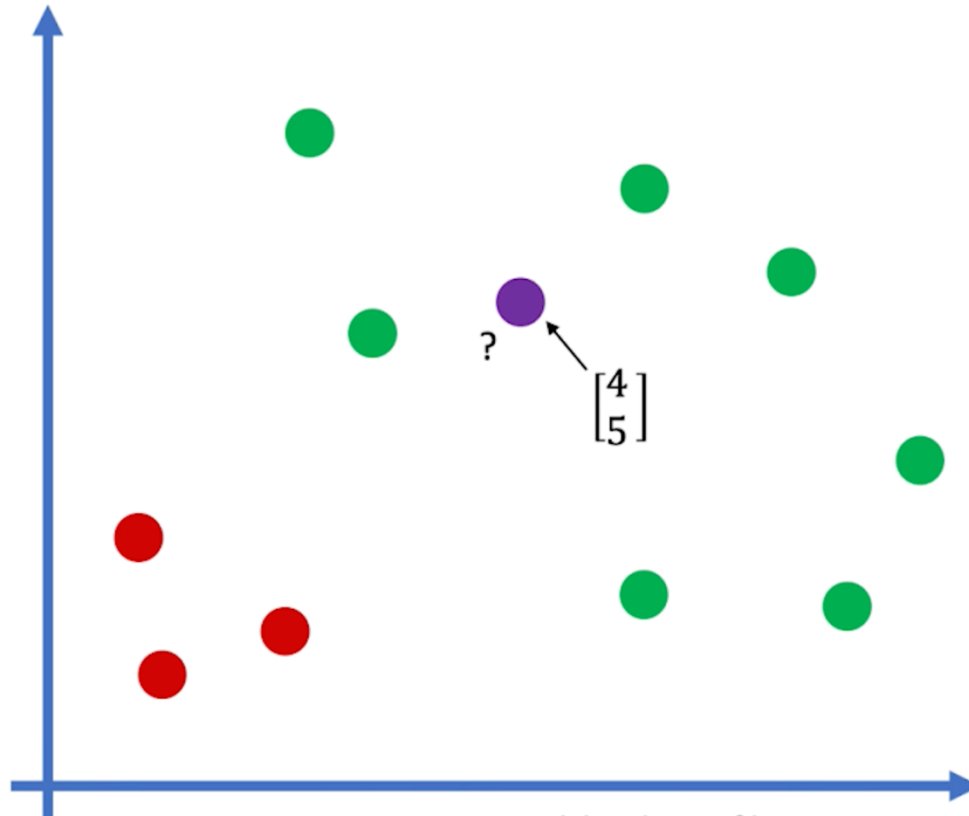
x_1 = *Número de clases a las que asistió*

x_2 = *Horas que dedicó a estudiar por su cuenta*



Problema de aplicación

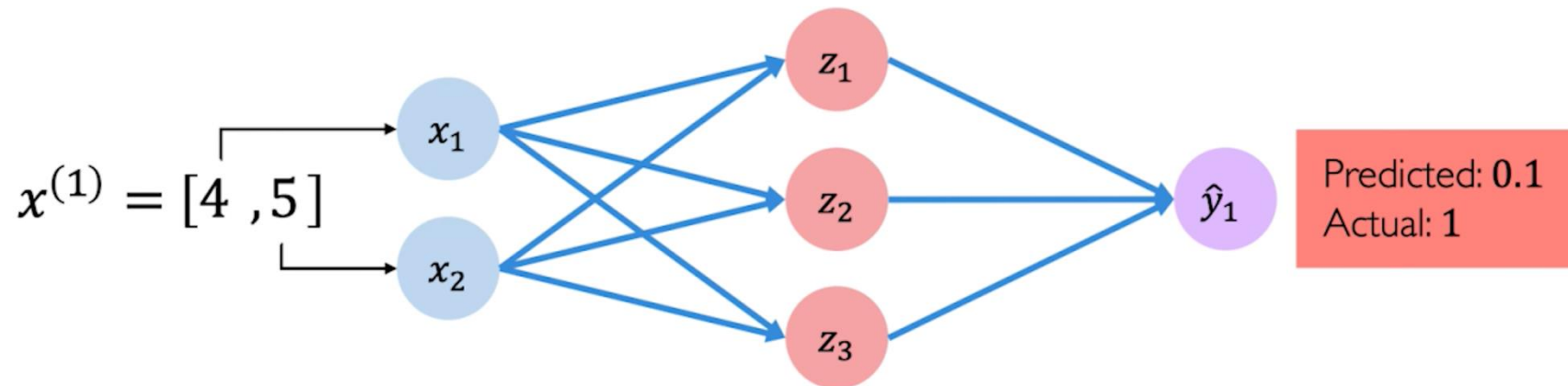
$x_1 = \text{Número de clases}$
 $x_2 = \text{Horas de estudio}$



Cuantificando pérdida

La **pérdida** (loss) en una red, mide el costo de hacer malas predicciones.

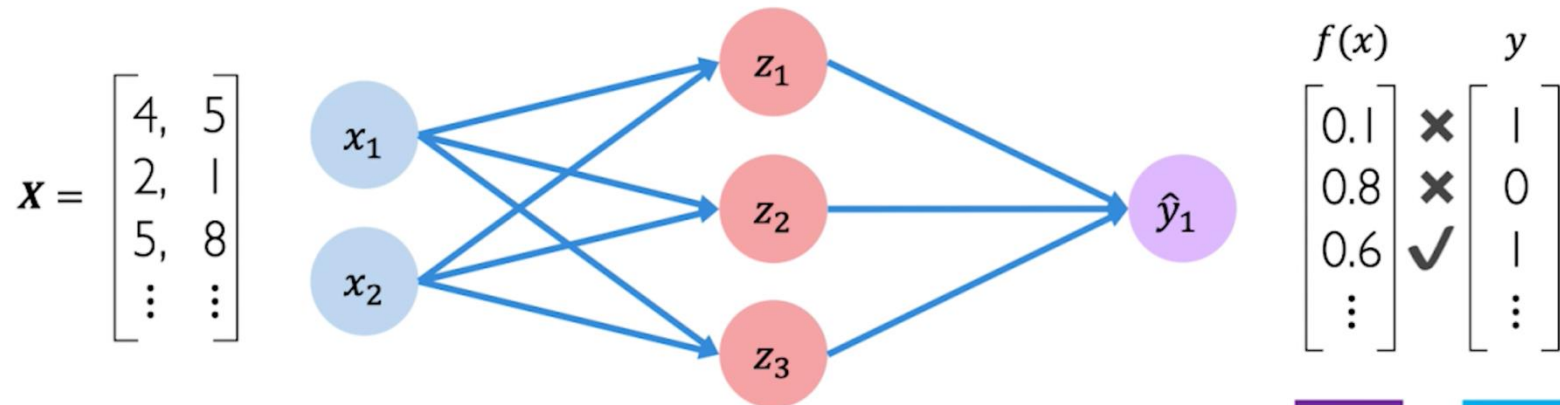
- Se trata de reducir la pérdida al máximo
- Pérdida pequeña => Valor predicho es muy parecido al real.



$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Función de Pérdida Empírica

La **pérdida empírica** (empirical loss) mide la pérdida total sobre un dataset completo.



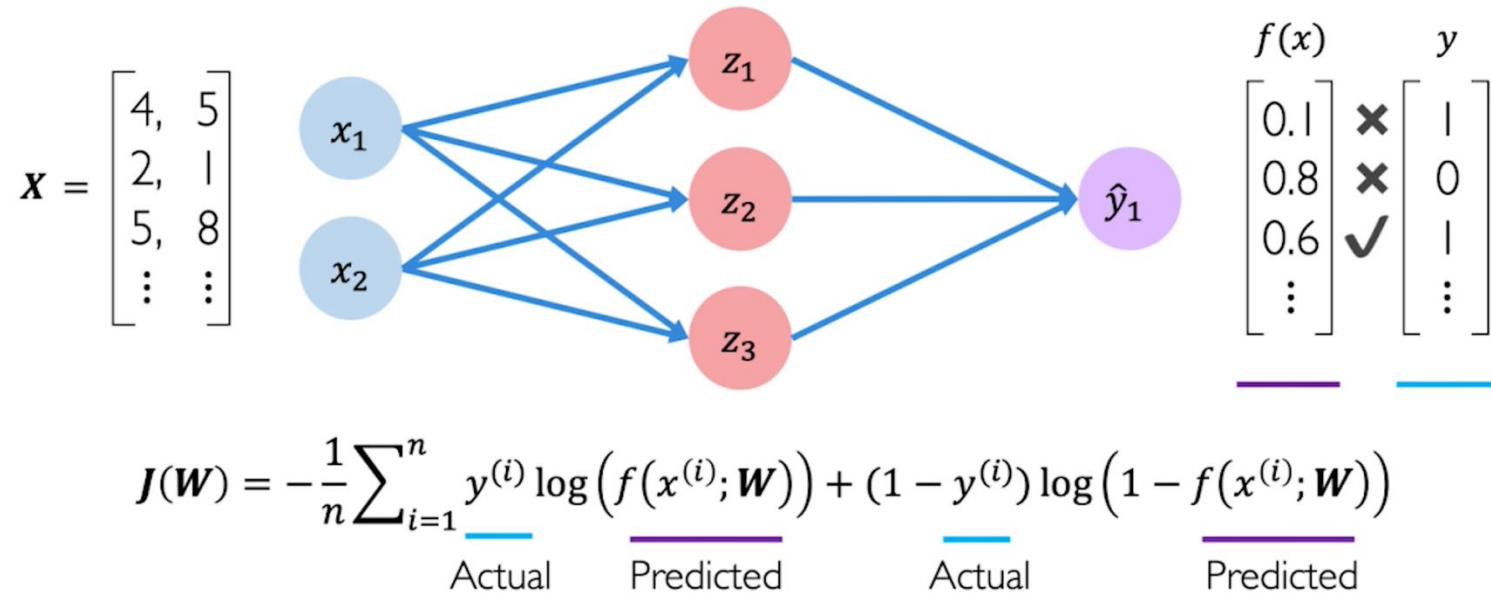
Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Pérdida de entropía cruzada binaria (Binary cross entropy loss)

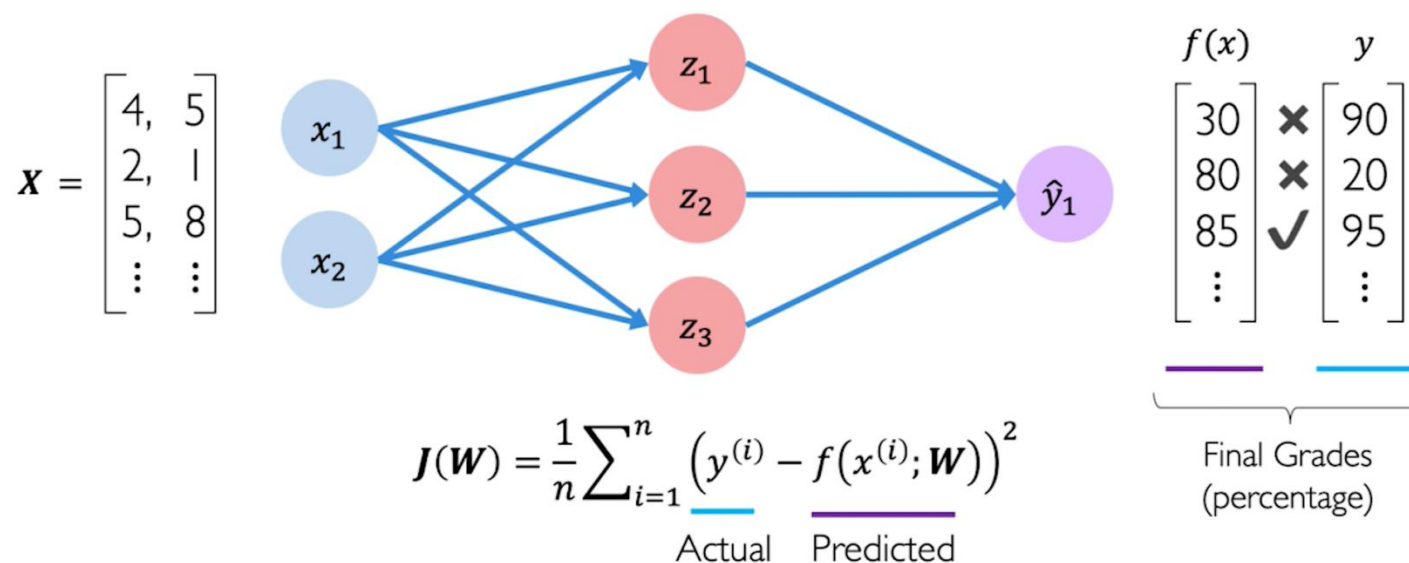
La **pérdida de entropía cruzada**, se puede utilizar en modelos que obtienen como salida una decisión binaria 0 / 1 (Si / No).



```
loss = tf.reduce_mean( tf.nn.softmax_cross_entropy_with_logits(y, predicted) )
```

Pérdida de Error Cuadrático Medio (Mean square error loss)

Este tipo de pérdida se puede emplear con modelos de regresión que obtienen como salida números reales continuos.

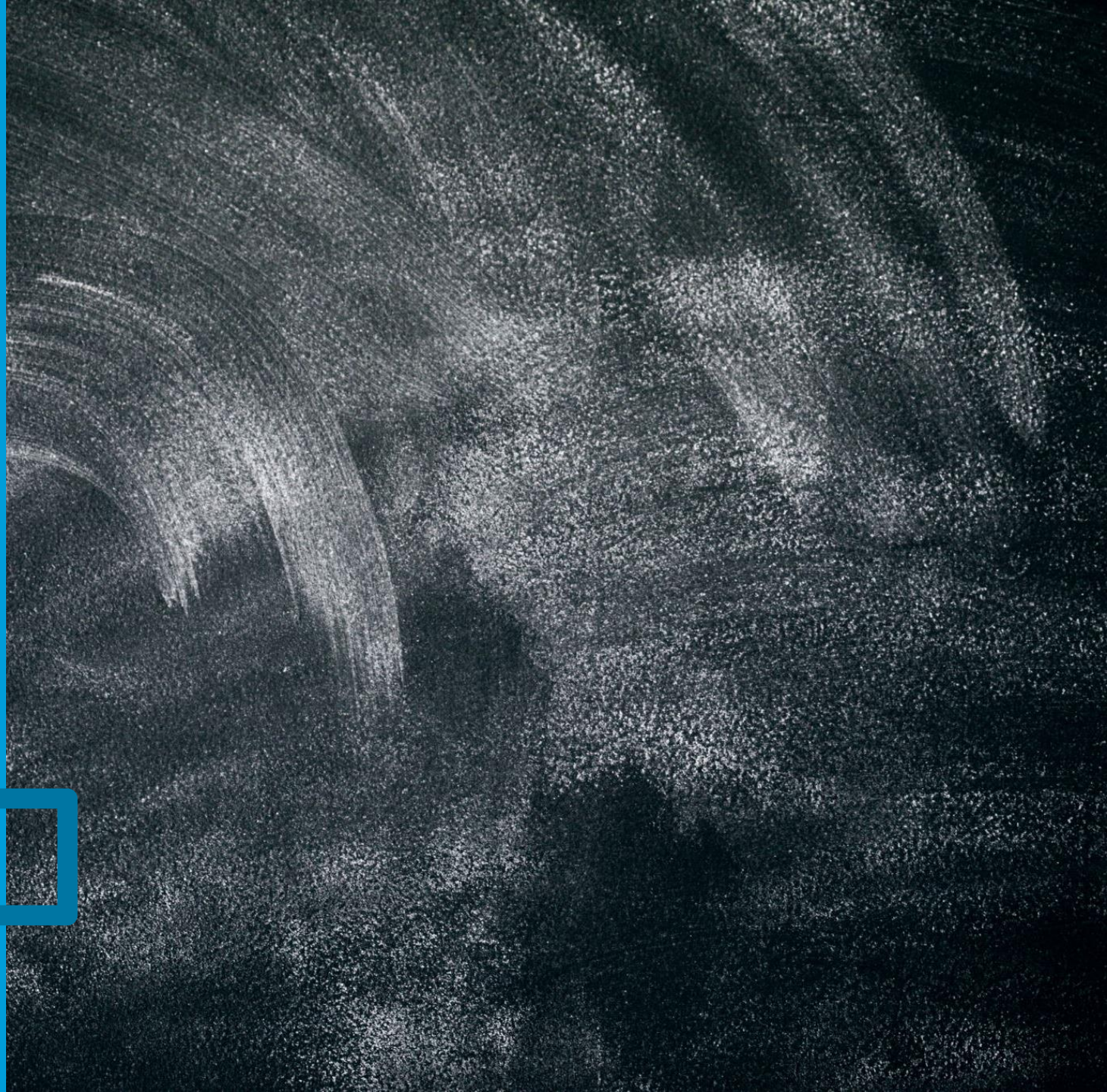


```
loss = tf.reduce_mean( tf.square(tf.subtract(y, predicted)) )  
loss = tf.keras.losses.MSE( y, predicted )
```



Deep Learning Redes Neuronales

Proceso de Entrenamiento



Optimización de Pérdida

Se requiere encontrar los pesos para obtener una pérdida mínima

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

Optimización de Pérdida

Se requiere encontrar los pesos para obtener una pérdida mínima

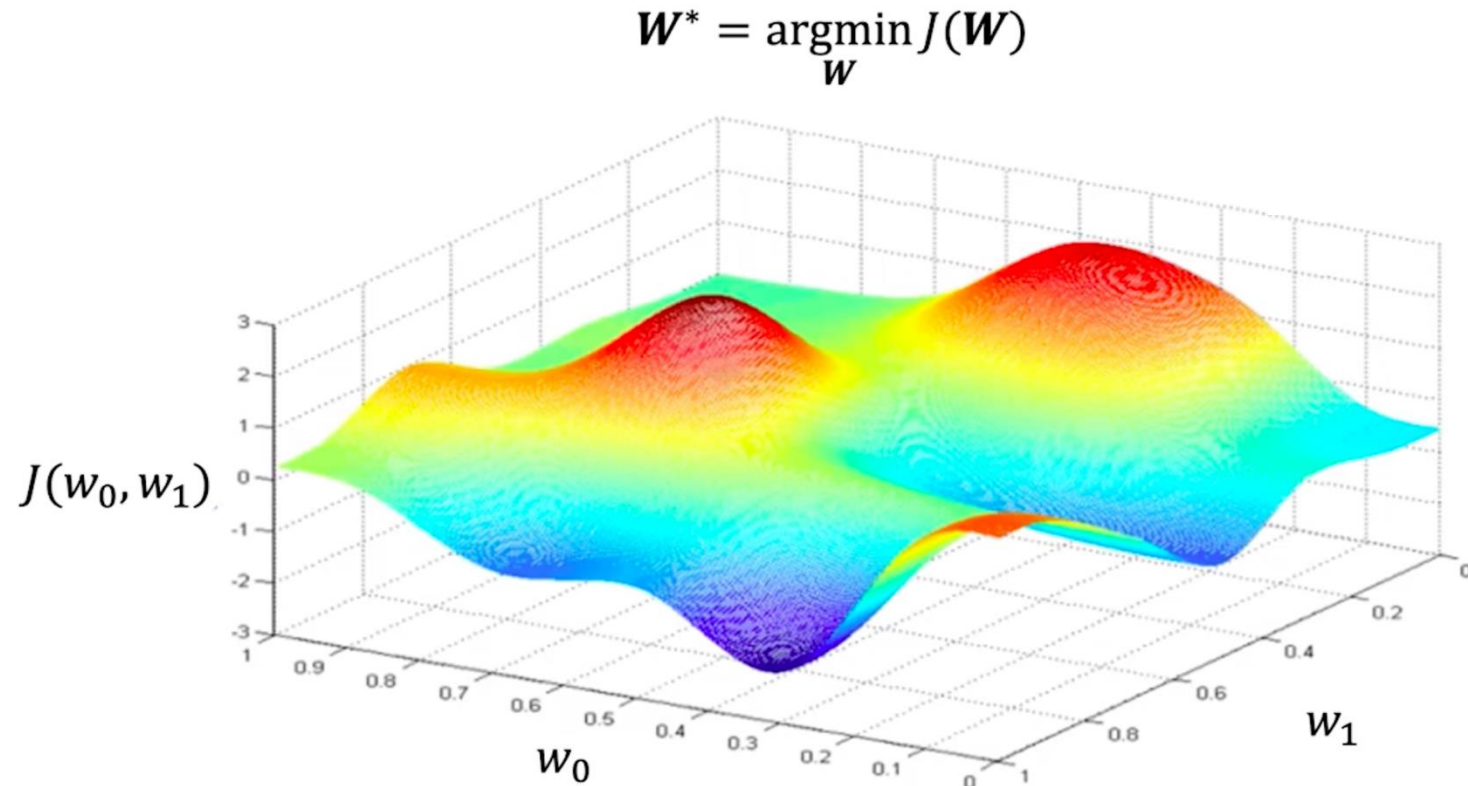
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

$$\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$$

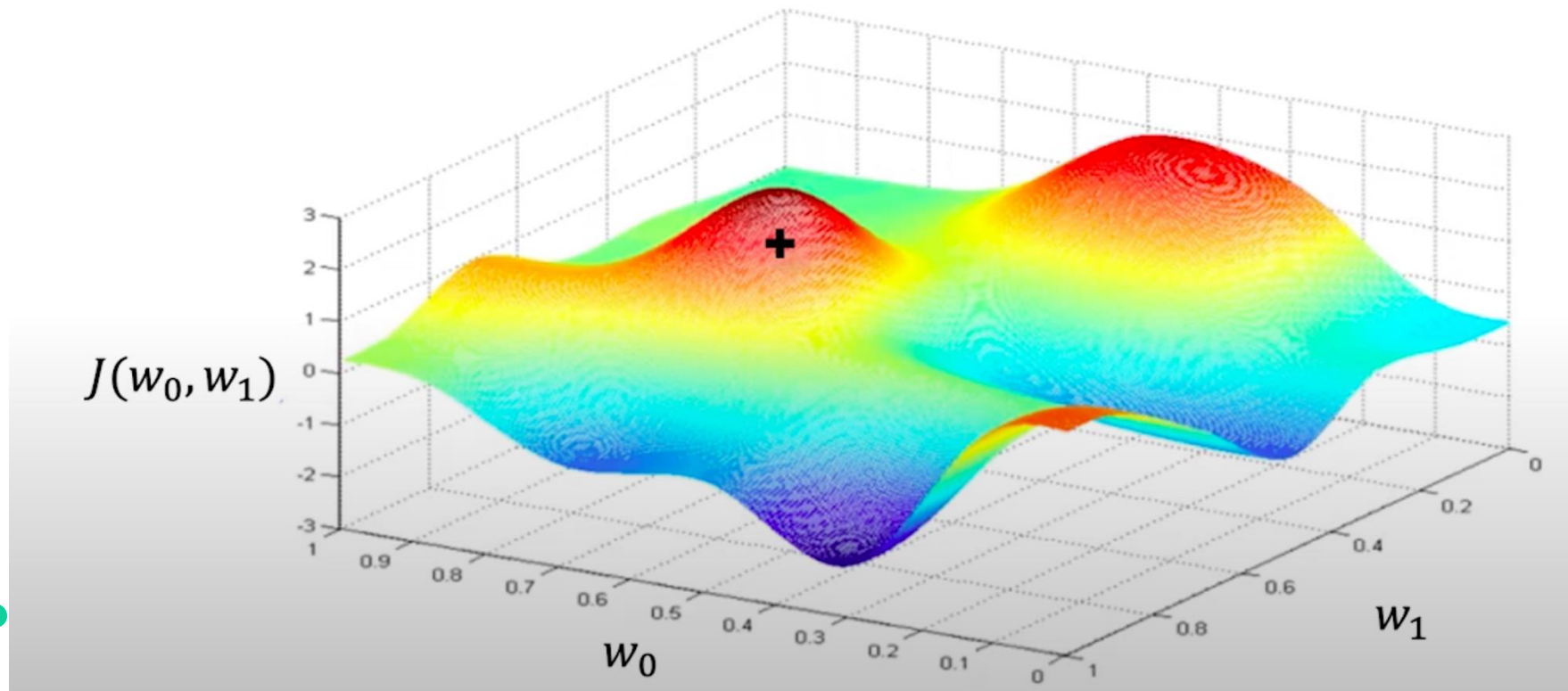
Optimización de Pérdida

Pérdida: Función que representa los pesos de la red



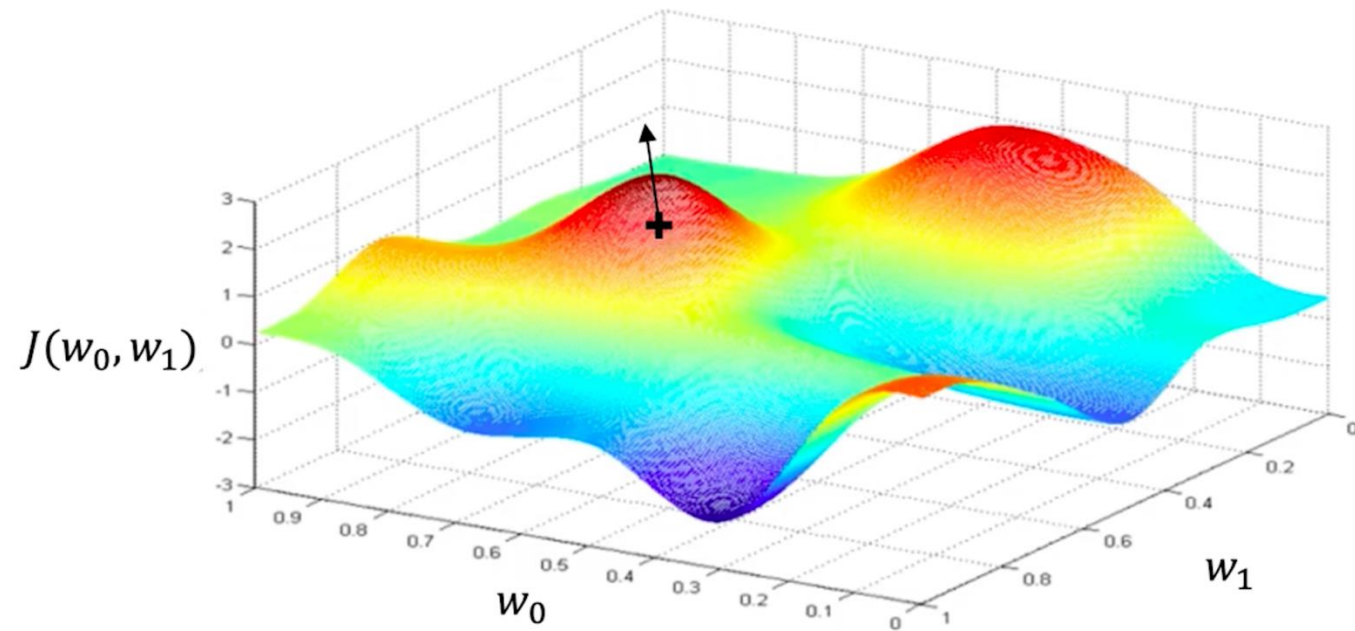
Optimización de Pérdida

Se inicializa un punto aleatorio (w_0, w_1)



Optimización de Pérdida

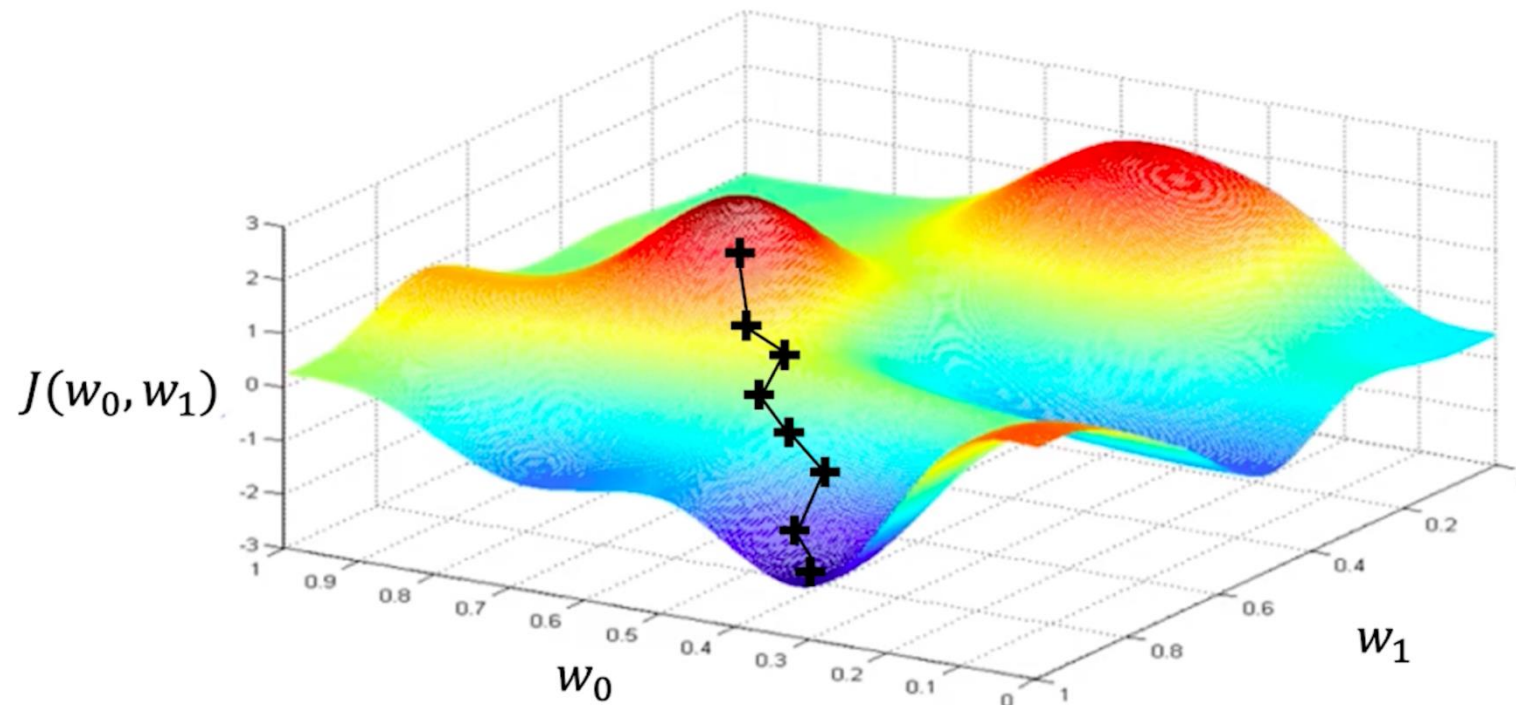
Se calcula el gradiente $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$



Descenso de gradiente (Gradient descent)

Se repite hasta que converja a un mínimo

Repeat until convergence



Descenso de gradiente (Gradient descent)

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

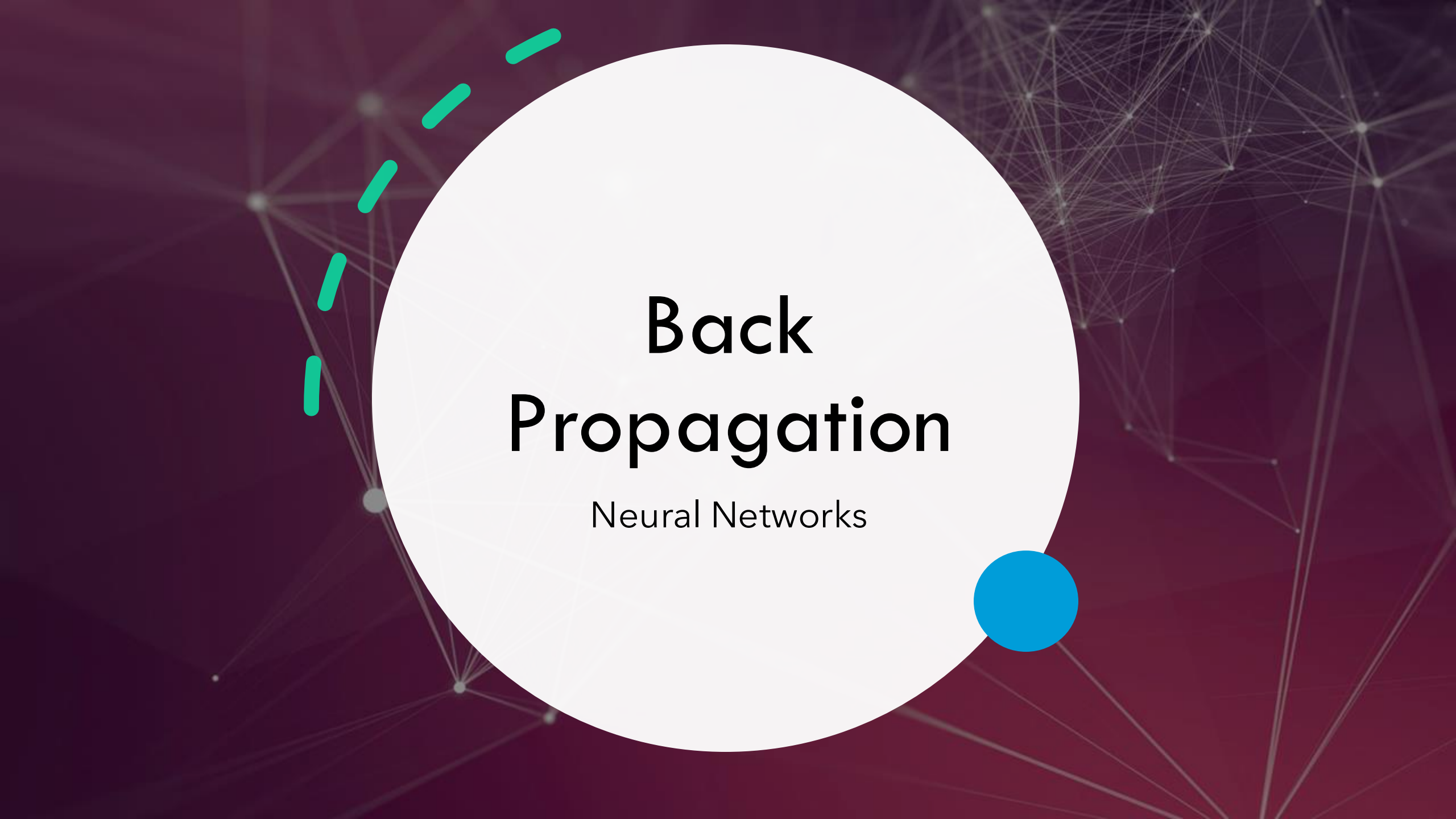


```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()] )

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

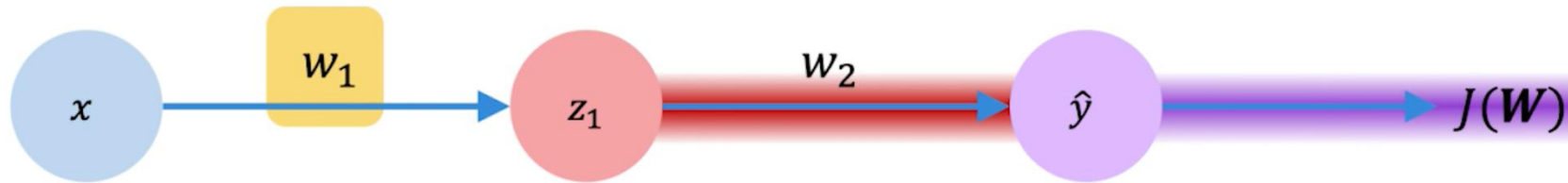
    weights = weights - lr * gradient
```



Back Propagation

Neural Networks

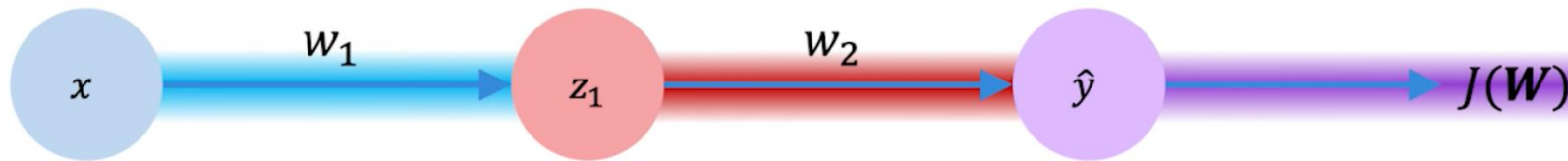
Calculando gradientes: Back Propagation



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

Apply chain rule! Apply chain rule!

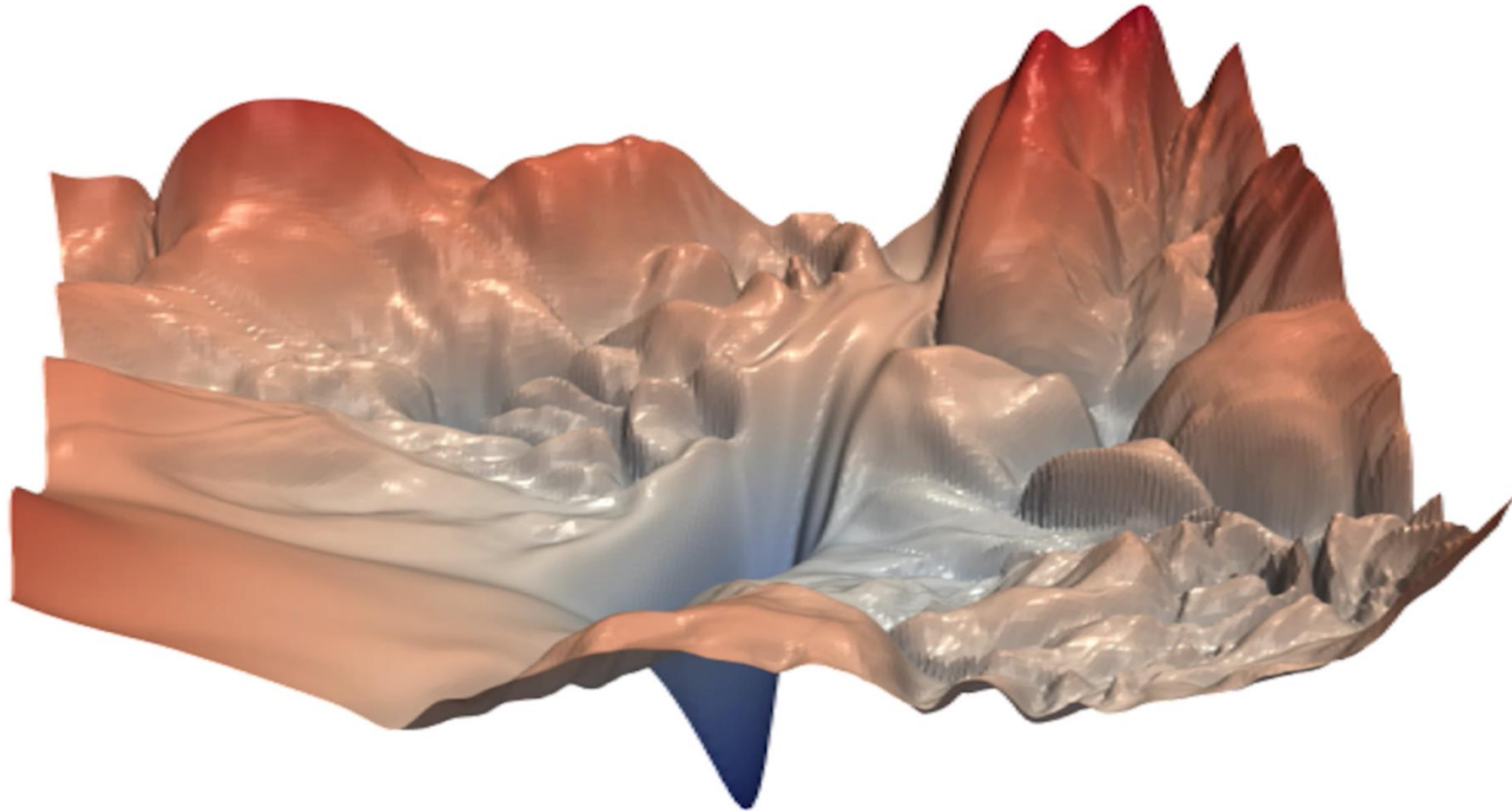
Calculando gradientes: Back propagation



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Se repite para cada peso en la red usando los gradientes de las capas siguientes

Entrenamiento de una Red Neuronal en la Práctica

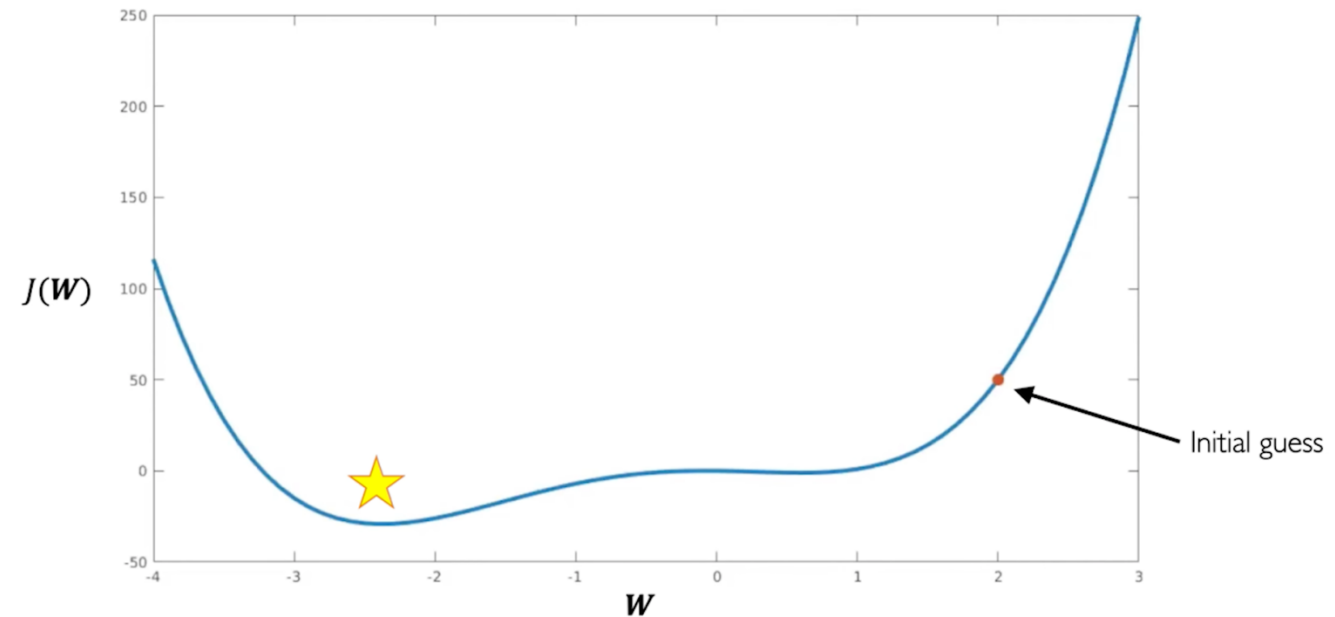


Función de pérdida puede ser difícil de optimizar

Optimización a través de método del gradiente

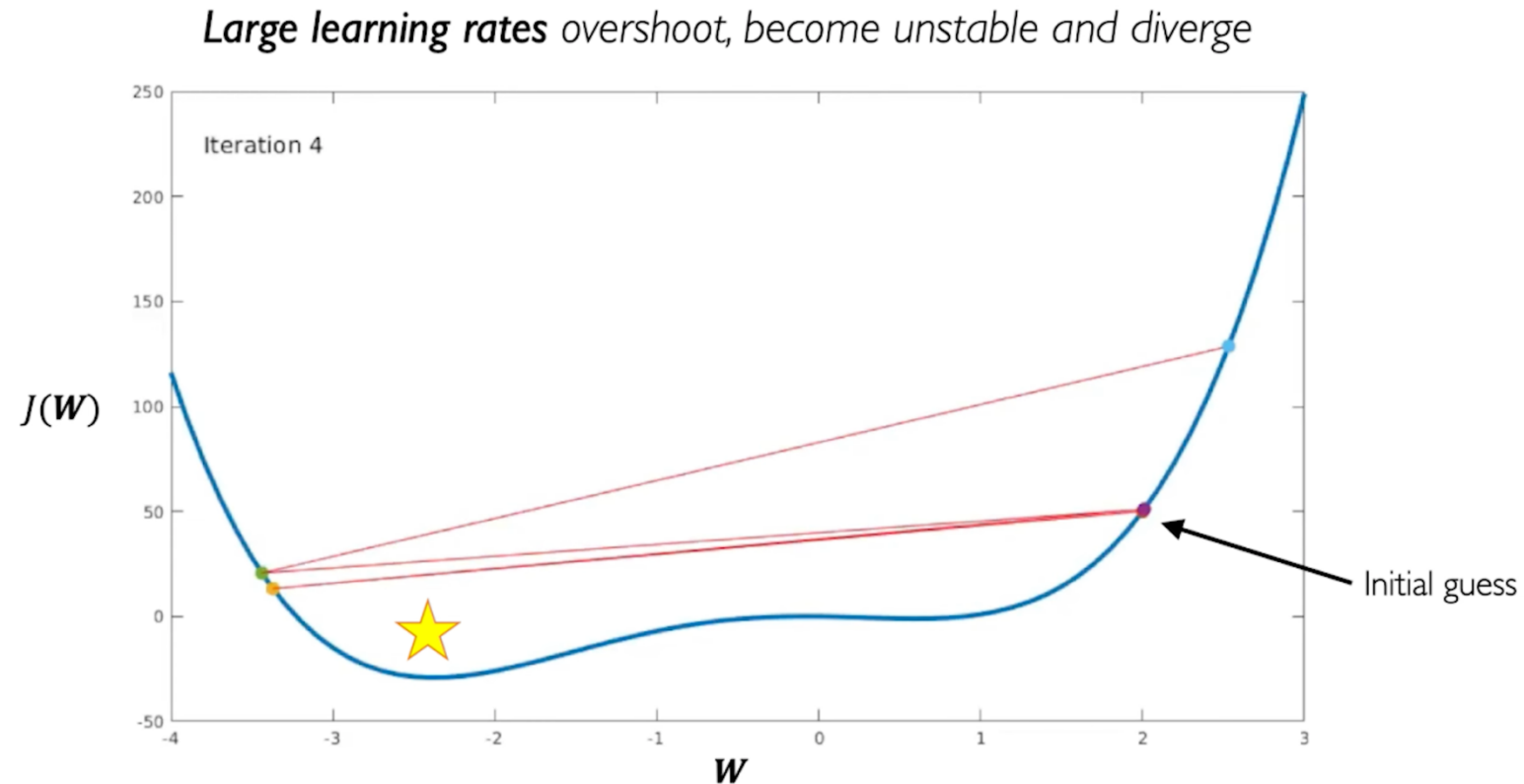
$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

↑
Tasa de aprendizaje
(learning rate)



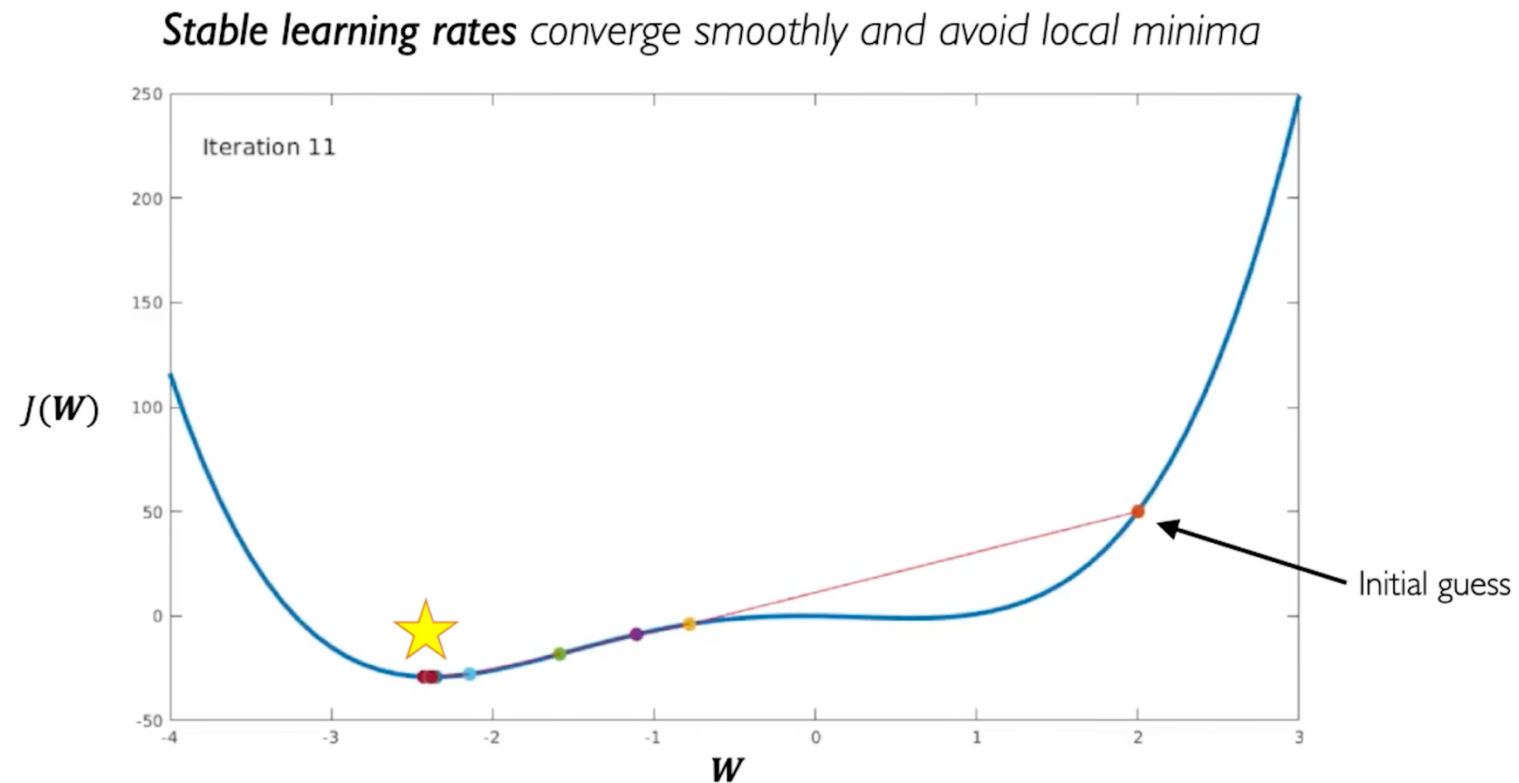
Función de pérdida puede ser difícil de optimizar

Optimización a través de método del gradiente



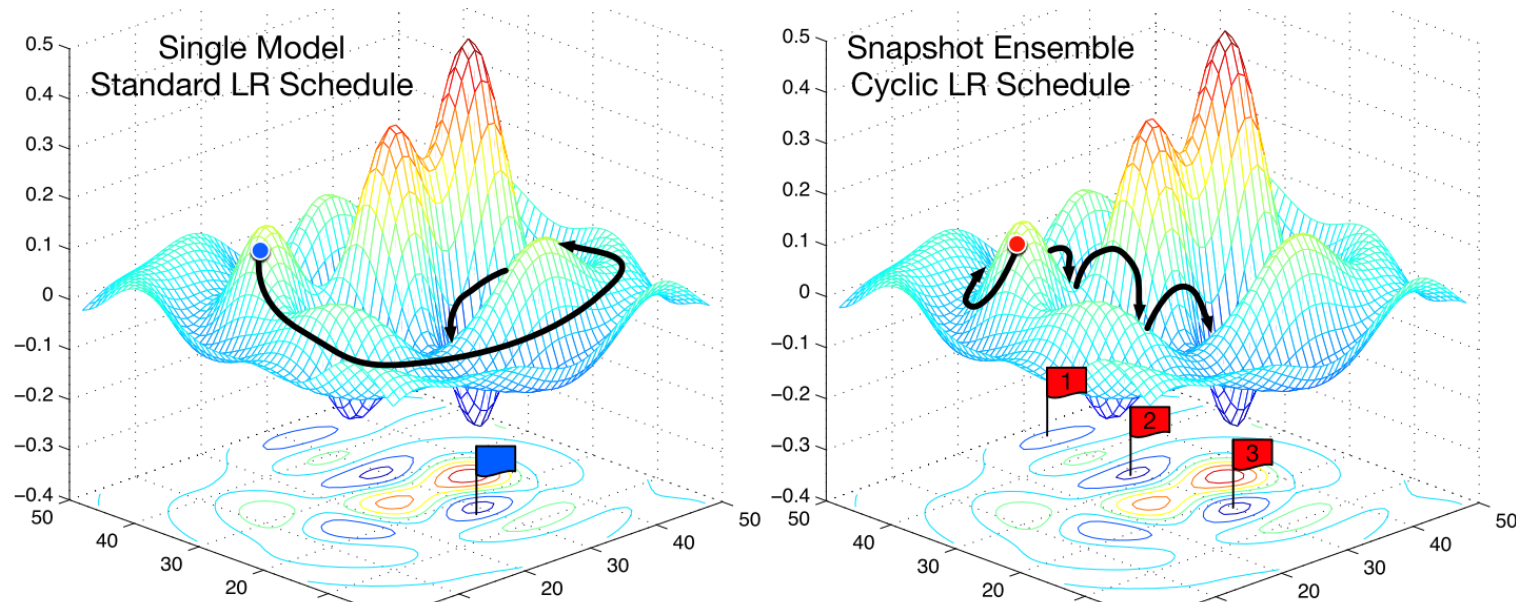
Función de pérdida puede ser difícil de optimizar

Optimización a través de método del gradiente



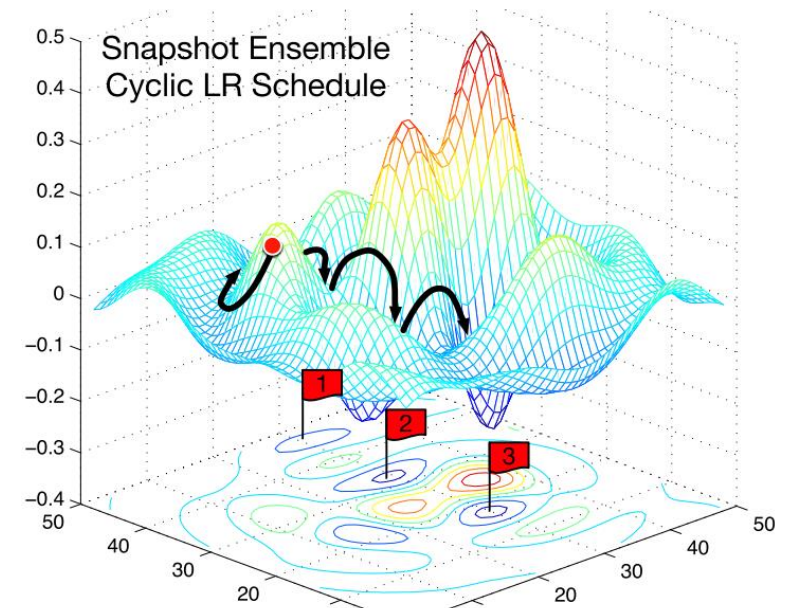
¿Cómo seleccionar un learning rate adecuado?

- **Enfoque 1:** Probar múltiples valores y seleccionar aquel (aquellos) que mejor funcione.



¿Cómo seleccionar un learning rate adecuado?

- **Enfoque 2:** Diseñar un algoritmo de learning-rate adaptativo.
 - Los learning rates no son fijos
 - Pueden ser más grandes o pequeños dependiendo de:
 - La magnitud del gradiente
 - Que tan rápido esta pasando el cambio
 - Magnitud de los pesos
 - Etc...



Algoritmos de Gradient Descent

Algorithm	TF Implementation	Reference
• SGD	 <code>tf.keras.optimizers.SGD</code>	Kiefer & Wolfowitz. "Stochastic Estimation of the Maximum of a Regression Function." 1952.
• Adam	 <code>tf.keras.optimizers.Adam</code>	Kingma et al. "Adam: A Method for Stochastic Optimization." 2014.
• Adadelata	 <code>tf.keras.optimizers.Adadelata</code>	Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.
• Adagrad	 <code>tf.keras.optimizers.Adagrad</code>	Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.
• RMSProp	 <code>tf.keras.optimizers.RMSProp</code>	

Diseñando una primer aplicación con Tensor Flow

```
import tensorflow as tf

model = tf.keras.Sequential([...])

# pick your favorite optimizer
optimizer = tf.keras.optimizer.SGD()

while True: # loop forever

    # forward pass through the network
    prediction = model(x)

    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)

    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

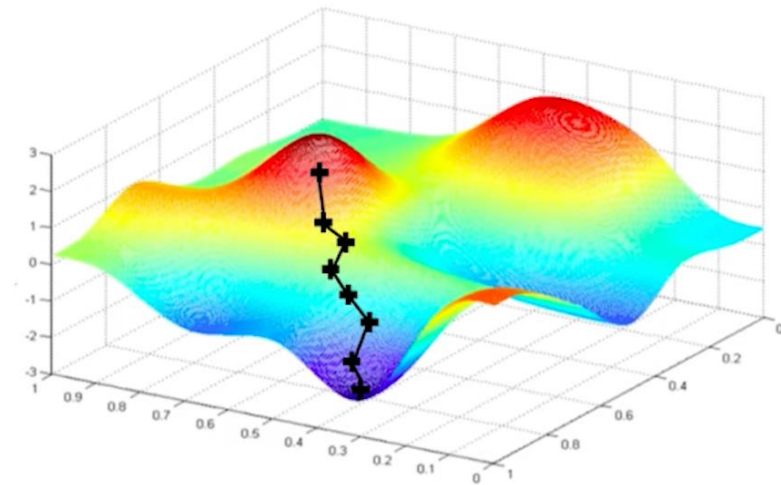


Can replace with any
TensorFlow optimizer!

Redes Neuronales en la práctica: Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights



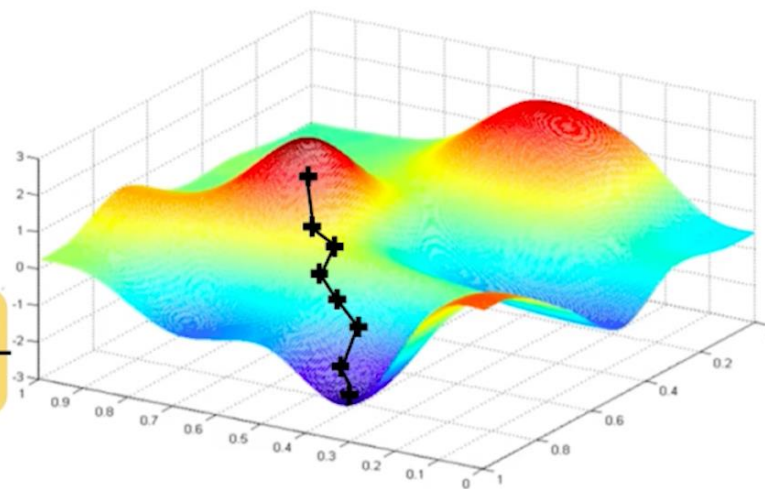
Can be very
computationally
intensive to compute!

Si se calcula sobre cada punto es computacionalmente muy caro.

Redes Neuronales en la práctica: Stochastic Gradient Descent (Mini-batches)

Algorithm

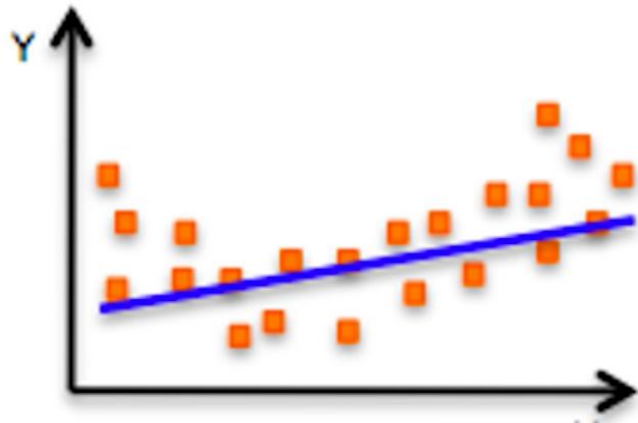
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Fast to compute and a much better
estimate of the true gradient!

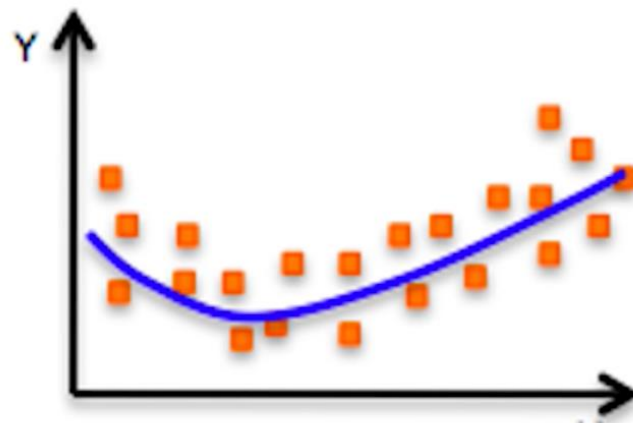
Mini batches -> Son del orden de cientos o miles de elementos.

El problema de Sobre-estimaciones

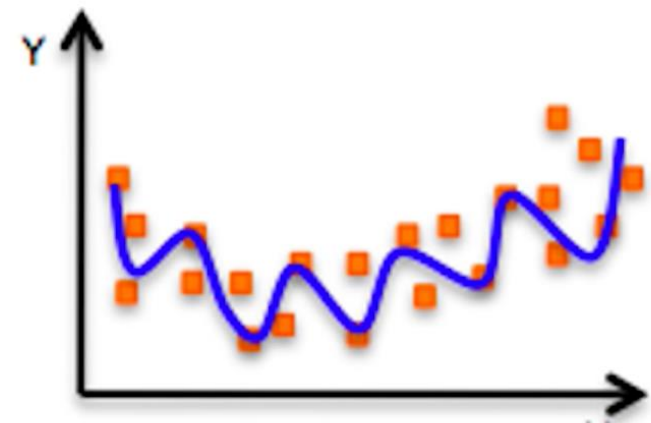


Underfitting

Model does not have capacity to fully learn the data



Ideal fit



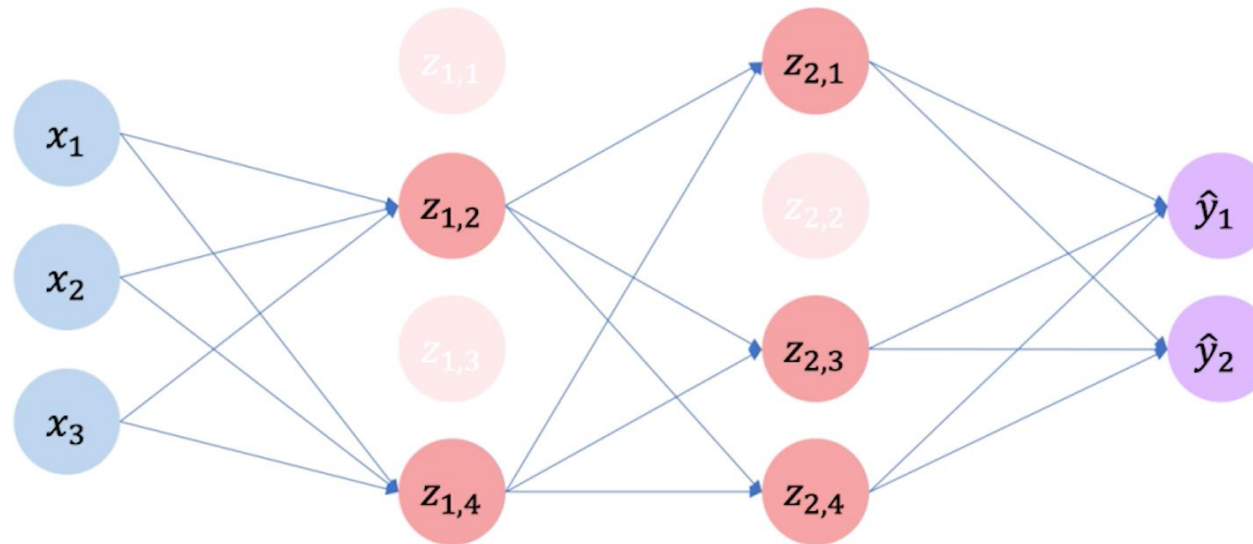
Overfitting

Too complex, extra parameters, does not generalize well

Regularización 1: Dropout

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

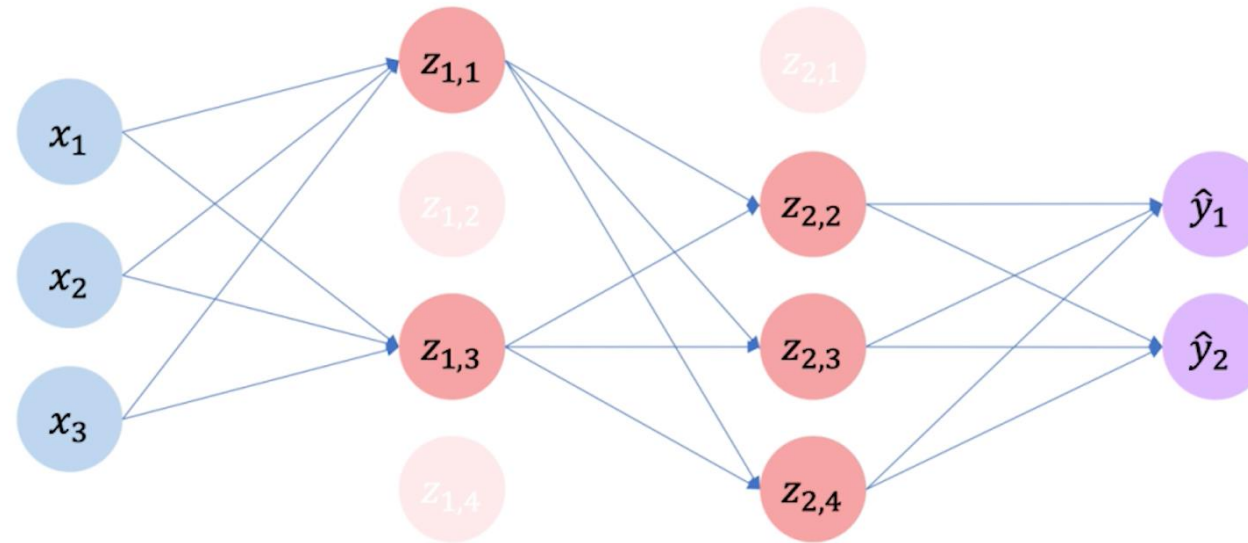
 `tf.keras.layers.Dropout(p=0.5)`



Regularización 1: Dropout

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

 `tf.keras.layers.Dropout(p=0.5)`



Regularización 2: Dropout

- Stop training before we have a chance to overfit

